Proceso Scrum

CivicTrackIO

Autores:
Luna Alexandra Beltrán Escobar
Sarah Esther Calderón Legarda
Leidy Fernanda Eraso Vásquez

DOCUMENTO	
Título:	PS-CivicTrackIO
Fecha de elaboración:	28 de marzo del 2026
Versión:	1

REGISTRO DE CAMBIOS			
Versión	*Páginas*	*Fecha de modificación*	*Motivo del cambio*

Contenido

1. Organización del proceso **4**
2. Desarrollo del proceso **4**
2.1. Sprint 1 *4*
2.1.1. Planeación del Sprint 1 4
2.1.2. Ejecución del Sprint 1 8
2.1.3. Evaluación 17
2.2. Sprint 2 *18*
2.2.1. Planeación del Sprint 2 18
2.2.2. Ejecución del Sprint 2 19
2.2.3. Evaluación 20
2.3. *Sprint 3* 20
2.3.1. Planeación del Sprint 3 20
2.3.2. Ejecución del Sprint 3 21
2.3.3. Evaluación 21
2.4. *Sprint 4* 22
2.4.1. Planeación del Sprint 4 22
2.4.2. Ejecución del Sprint 4 23
2.4.3. Evaluación 23

1. Organización del proceso

Para la gestión del proceso de desarrollo de la solución computacional, se utilizó *Scrum*. En total se desarrollarán *4 Sprints*. Como actividad previa a la ejecución de las iteraciones e incrementos, se identificaron, especificaron y priorizaron las necesidades de los interesados como se puede observar en la plantilla para la gestión del *Product Backlog*. Para establecer las necesidades, se utilizó como técnica la historia de usuario. Cada Sprint tuvo una duración de dos (2) semanas.

El *Product Goal* establecido por el equipo es: *Desarrollar una plataforma que permita registrar hurtos georreferenciados y visualizar zonas de riesgo mediante un mapa de calor y estadísticas confiables, para apoyar la prevención y la toma de decisiones seguras de los ciudadanos.*

Cada *Sprint* se encuentra estructurado por la planeación, desarrollo y evaluación.

2. Desarrollo del proceso

El proyecto desarrollado, junto con su estructura de carpetas, código fuente y evidencias relacionadas, se encuentra almacenado en un repositorio público/privado de GitHub. Este repositorio permite consultar el avance del desarrollo, así como los cambios realizados mediante el historial de commits. El acceso al proyecto se puede realizar a través del siguiente enlace: https://github.com/dreamylullaby/civictrackio_project. A continuación, se presenta la información recopilada en la ejecución de cada *Sprint.*

2.1. Sprint 1

2.1.1. Planeación del *Sprint 1*

Con base en las necesidades identificadas y priorizadas, en el *Sprint Planning Meeting* se seleccionaron las historias de usuario a realizar en el *Sprint*, las cuales se pueden observar en la Tabla 1.

Tabla 1. Sprint backlog

Id	Descripción de la Historia de Usuario	Prioridad	Complejidad
HU1	Como usuario víctima o testigo de un hurto quiero registrar un incidente indicando fecha, franja horaria, ubicación, barrio y tipo de hurto para contribuir al mapa colaborativo.	90	Alta
HU2	Como Usuario/Ciudadano, quiero iniciar sesión ingresando mi correo electrónico,	80	Alta

	para acceder al sistema de manera rápida y sencilla.		
HU3	Como administrador quiero iniciar sesión con credenciales seguras y rol administrativo, para acceder al panel de gestión del sistema.	80	Alta
HU4	Como usuario, quiero registrarme ingresando mi correo electrónico/apodo y una contraseña válida, para crear una cuenta y acceder a las funcionalidades del sistema.	80	Alta
HU5	Como usuario (administrador o ciudadano), quiero cerrar sesión en la aplicación, para proteger mi información y evitar accesos no autorizados.	64	Baja
HU6	Como usuario quiero especificar el objeto hurtado, número de agresores y añadir una breve descripción para aportar mayor contexto al incidente registrado.	63	Media

El *Sprint Goal* establecido fue: *Implementar las funcionalidades base de autenticación y registro de hurtos para habilitar el acceso seguro al sistema y la recolección inicial de incidentes georreferenciados.*

En la Tabla 2, se puede observar el *Definition of Done* (DoF) establecido para este Sprint con las personas responsables por cada tarea.

Tabla 2. Definition of Done del Sprint

Id-HU	**Tarea**	**Responsable**	**Tiempo estimado (HH:MM)**	**Tiempo realizado (HH:MM)**
HU1	Especificar la HU-01	**Luna**	00:30	00:30
	Definir tabla reportes con tipo, fecha, hora, ubicación, zona y relación con usuarios.	**Sarah**	02:00	04:00
	Crear migración SQL para crear la tabla.	**Sarah**	00:30	00:30
	Definir endpoint POST/reportes y estructura del modelo/DAO.	**Fernanda**	06:00	06:00
	Implementar Endpoint POST /reportes que valide datos obligatorios y registre el incidente con manejo de errores.	**Sarah**	04:00	05:00

	Pantalla/formulario “Registrar hurto” con tipo, fecha, hora, ubicación.	**Sarah**	02:00	02:30
	Implementar validaciones de campos y llamada al endpoint; mostrar mensajes de éxito/error.	**Fernanda**	04:00	04:00
	Identificar escenarios de prueba	**Luna**	01:00	00:30
	Diseñar y crear casos de prueba	**Luna**	02:00	00:30
	Ejecutar casos de prueba	**Luna**	01:00	
	Realizar ajustes con base en las pruebas	**Sarah**	01:00	01:00
HU2	Especificar la HU-02	**Luna**	0:30	0:20
	Definir tabla usuarios con correo, hash_password, rol admin o ciudadano (por defecto ciudadano), estado y fecha de creacion.	**Sarah**	03:00	04:00
	Migración para crear tabla e índice único en correo.	**Sarah**	0:30	0:15
	Definir Endpoint POST /auth/login, estructura del JWT, manejo de errores.	**Fernanda**	06:00	04:00
	Implementar POST /auth/login con validación de correo, búsqueda de usuario y verificación de contraseña, generación de JWT.	**Fernanda**	05:00	04:00
	Pantalla de login con correo y contraseña o autenticacion con google, validación básica.	**Sarah**	04:00	06:00
	Consumir POST /auth/login, guardar token y navegar a la pantalla principal.	**Fernanda**	06:00	04:00
	Identificar escenarios de prueba	**Luna**	01:00	0:10
	Diseñar y crear casos de prueba	**Luna**	02:00	0:30
	Ejecutar casos de prueba	**Luna**	01:00	03:00
	Realizar ajustes con base en las pruebas	**Sarah**	01:00	01:00
HU3	Especificar la HU-03	**Luna**	00:30	00:10
	Definir POST /auth/admin-login, middlewares de autenticación y de rol admin, y rutas protegidas.	**Fernanda**	06:00	04:00
	Implementar POST /auth/admin-login reutilizando la	**Fernanda**	05:00	04:00

	lógica de login, pero verificando rol; crear middlewares auth y admin.			
	Pantalla de login admin con validación de correo.	**Sarah**	02:00	01:30
	Consumir POST /auth/admin-login, guardar token, redirigir al panel y rutas protegidas..	**Fernanda**	05:00	04:00
	Identificar escenarios de prueba	**Luna**	01:00	00:10
	Diseñar y crear casos de prueba	**Luna**	02:00	00:20
	Ejecutar casos de prueba	**Luna**	01:00	01:00
	Realizar ajustes con base en las pruebas	**Sarah**	01:00	01:00
HU4	Especificar la HU-04	**Luna**	00:30	00:15
	Definir Endpoint POST /auth/register, reglas de validación y mensajes de error.	**Fernanda**	05:00	04:00
	Implementar endpoint de registro: validar, revisar duplicados, hashear contraseña y crear usuario.	**Fernanda**	06:00	04:00
	Pantalla “Registrarse” con correo, contraseña y confirmación.	**Sarah**	02:00	02:30
	Implementar pantalla de registro en Flutter, validaciones de campos, llamada a /auth/register y redirección a login tras éxito.	**Fernanda**	06:00	04:00
	Identificar escenarios de prueba	**Luna**	01:00	00:10
	Diseñar y crear casos de prueba	**Luna**	02:00	00:20
	Ejecutar casos de prueba	**Luna**	01:00	01:30
	Realizar ajustes con base en las pruebas	**Sarah**	01:00	01:00
HU5	Especificar la HU-05	**Luna**	00:30	00:12
	Definir el endpoint POST /auth/logout para manejar el cierre de sesión e invalidar el token.	**Fernanda**	04:00	04:00
	Implementar el endpoint y eliminar el token tanto en el servidor como en el cliente.	**Fernanda**	05:00	04:00
	Botón “Cerrar sesión” en el panel; al pulsar: borrar token, llamar opcionalmente a /auth/logout y redirigir a login admin.	**Fernanda**	04:00	04:00
	Opción “Cerrar sesión” en menú; al pulsar: borrar token almacenado y volver a pantalla de login.	**Sarah**	01:00	01:30
	Identificar escenarios de prueba	**Luna**	01:00	00:10

	Diseñar y crear casos de prueba	**Luna**	02:00	00:20
	Ejecutar casos de prueba	**Luna**	01:00	03:00
	Realizar ajustes con base en las pruebas	**Sarah**	01:00	01:00
HU6	Especificar la HU-06	**Luna**	00:30	00:20
	Añadir campos objeto hurtado, número de agresores y descripción a reportes (máx. 300 caracteres).	**Sarah**	00:15	00:15
	Migración para alterar la tabla si no estaban definidos.	**Sarah**	00:30	00:15
	Modificar POST /reportes para guardar objeto hurtado, número de agresores y descripción, verificar control de errores.	**Fernanda**	06:00	05:00
	Ampliar formulario de registro de hurto con objeto hurtado, número de agresores y descripción, y enviar estos campos al endpoint.	**Fernanda**	06:00	06:00
	Identificar escenarios de prueba	**Luna**	01:00	00:10
	Diseñar y crear casos de prueba	**Luna**	02:00	00:20
	Ejecutar casos de prueba	**Luna**	01:00	02:00
	Realizar ajustes con base en las pruebas	**Sarah**	01:00	01:00

2.1.2. Ejecución del *Sprint 1*

Durante el Sprint 1 se elaboró el diagrama de arquitectura Clean Architecture del sistema CivicTrackIO, con el fin de definir la estructura general del software y organizar sus componentes de acuerdo con buenas prácticas de desarrollo y separación de responsabilidades. Este diagrama permitió identificar claramente las capas del sistema, diferenciando la capa de presentación (Frontend Flutter y React), la capa de aplicación (casos de uso y lógica de negocio), la capa de dominio (entidades y reglas centrales) y la capa de infraestructura (base de datos Supabase/PostgreSQL y servicios externos como autenticación y JWT). Asimismo, se documentaron las principales interacciones entre módulos, facilitando la comprensión del flujo de información entre el usuario, el backend y la base de datos. En el siguiente enlace se presenta el diagrama correspondiente a la arquitectura implementada para el Sprint 1. https://canva.link/7ano0wnfs4dx7c6

A continuación, se presenta una síntesis de los resultados obtenidos en la ejecución de las tareas establecidas para el *DoD*.

- **Especificación de la Historia de Usuario**

En el enlace indicado ESPECIFICACIÓN 1, se puede observar la especificación de las historias de usuario seleccionadas para el *Sprint*. En la Tabla 3, se presenta a manera de ejemplo una de las especificaciones.

Tabla 3. Ejemplo de la especificación de la HU-01

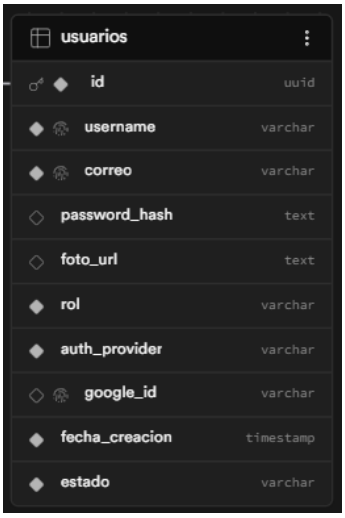
Historia de Usuario			
Código:	**HU-01**		
Nombre:	**Registrar incidente de hurto**		
Actor:	**Usuario / Víctima / Testigo**		
Descripción	Como usuario víctima o testigo de un hurto quiero registrar un incidente indicando fecha, franja horaria, ubicación, barrio y tipo de hurto para contribuir al mapa colaborativo.		
Criterios de aceptación:	**CID**	**Condición**	**Resultado**
	1	El usuario accede al formulario de registro e ingresa fecha, franja horaria, ubicación (GPS), barrio y tipo de hurto	El sistema valida que todos los campos obligatorios estén completos y habilita el envío
	2	El usuario intenta enviar el formulario con uno o más campos vacíos	El sistema impide el envío y muestra mensajes de validación específicos por campo
	3	El incidente se registra correctamente	El sistema muestra mensaje de confirmación al usuario
	4	Se registra un nuevo incidente mientras otro usuario tiene el mapa abierto	El sistema actualiza el mapa automáticamente mostrando el nuevo marcador
	5	El usuario ingresa una ubicación mediante GPS	El sistema guarda correctamente las coordenadas geográficas

- **Diseño e implementación de la base de datos**

Durante el Sprint 1 se diseñaron e implementaron temporalmente en Supabase las estructuras de base de datos necesarias para soportar la autenticación de usuarios y el registro de incidentes de hurto. Se crearon las tablas **usuarios**, **zonas** y **reportes**, incluyendo claves primarias y foráneas para garantizar la integridad de los datos. En la **Figura 1** se presenta el esquema de la tabla **usuarios** y el código SQL correspondiente para

su creación. En el enlace indicado BASE DE DATOS 1 Se puede consultar el diagrama de clases y el código completo de todas las tablas implementadas durante el Sprint 1.

Figura 1. Tabla usuarios




```
-- ============================================================

-- TABLA: usuarios

-- Usuarios registrados (local o Google)

-- ============================================================

CREATE TABLE public.usuarios (

    id              UUID        PRIMARY KEY DEFAULT gen_random_uuid(),

    username        VARCHAR(50) UNIQUE,

    correo          VARCHAR(150) NOT NULL UNIQUE,

    password_hash   TEXT,

    foto_url        TEXT,

    rol             VARCHAR(20) NOT NULL,

    auth_provider   VARCHAR(20) NOT NULL,

    google_id       VARCHAR(255) UNIQUE,

    fecha_creacion  TIMESTAMP   NOT NULL DEFAULT now(),
```

estado VARCHAR(20) NOT NULL,

CONSTRAINT usuarios_rol_check CHECK (rol IN ('usuario', 'admin')),

CONSTRAINT usuarios_auth_provider_check CHECK (auth_provider IN ('local', 'google')),

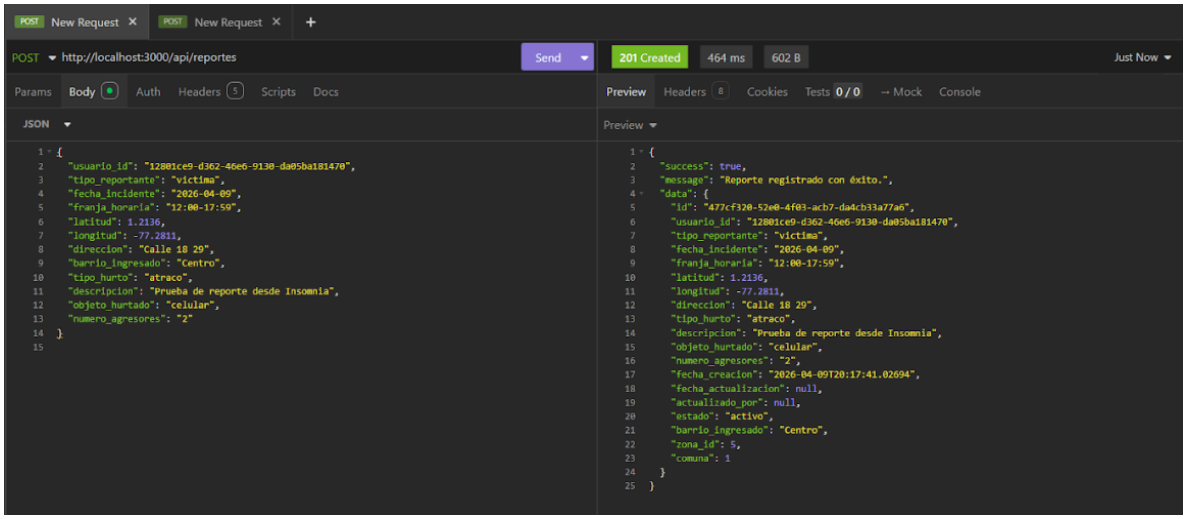
CONSTRAINT usuarios_estado_check CHECK (estado IN ('activo', 'bloqueado'))

);

- **Desarrollo del Back-end**

Se definieron e implementaron los endpoints correspondientes al Sprint 1, permitiendo el registro de usuarios, autenticación, cierre de sesión e inserción de reportes de hurto. Se incluyeron validaciones de campos obligatorios y manejo de errores. En la figura 2 se presenta la prueba del funcionamiento del endpoint POST /reportes. En el enlace indicado BACKEND 1 se encuentran las demás evidencias de las pruebas de los endpoints para el Sprint 1.

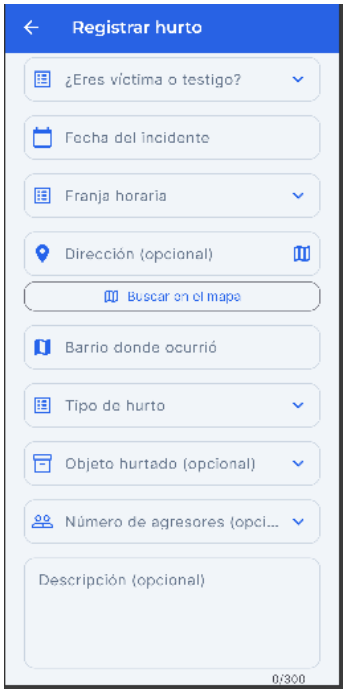
Figura 2. Funcionamiento endpoint reporte



- **Diseño de la Interfaz**
  - **Desarrollo del Front-end Flutter (Usuario)**

Se implementan las interfaces móviles correspondientes al Sprint 1: login, registro de usuario, registro de hurto y cierre de sesión. Las pantallas consumen los endpoints del backend y muestran mensajes de éxito o error según la respuesta del sistema. En la Figura 3, se presenta el diseño de la interfaz correspondiente al registro de hurto (HU-01). En el enlace indicado FRONTEND MOVIL 1 se encuentran las demás pantallas diseñadas para login, registro y cierre de sesión del usuario.

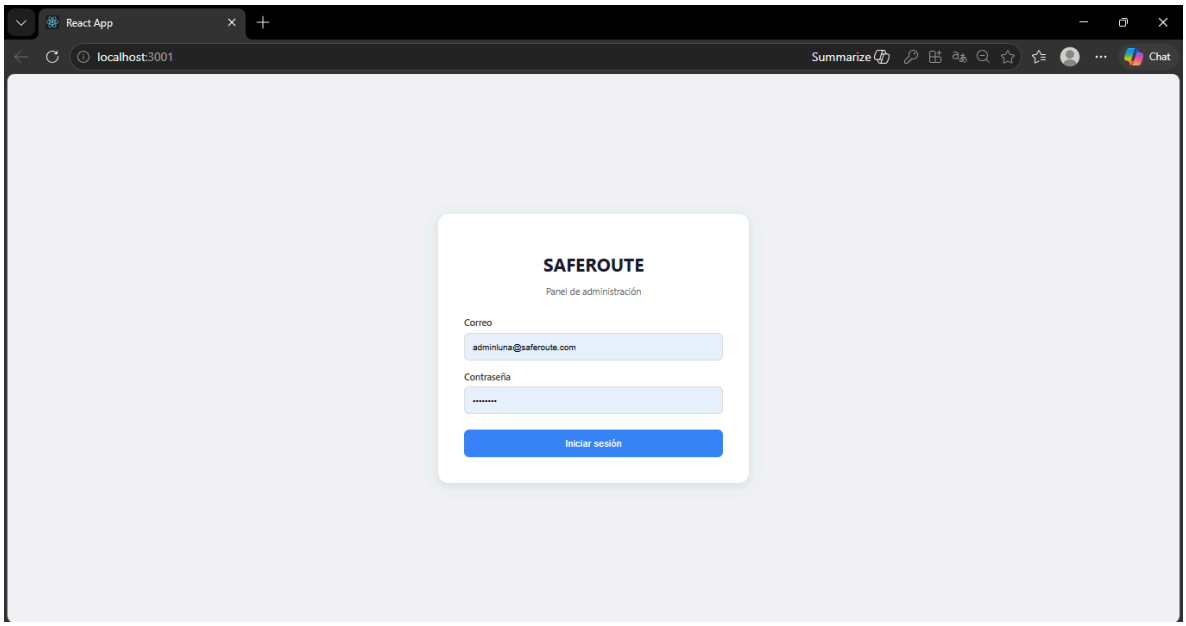
Figura 3. Pantalla "HU-01 Registrar hurto" (Flutter)



- **Desarrollo del Front-end React (Administrador)**

Se implementó el login administrativo y el cierre de sesión para el panel web. Además, se configuraron rutas protegidas para restringir el acceso únicamente a usuarios con rol administrador. En la Figura 4, se presenta el diseño de la interfaz correspondiente al inicio de sesión del administrador (HU-03). En el enlace indicado FRONTEND WEB 1 se encuentran las demás pantallas diseñadas para login y cierre de sesión del administrador.

Figura 4. Login administrador



- **Pruebas y Validación**

Se diseñaron y ejecutaron escenarios de prueba para las funcionalidades del Sprint 1, verificando casos exitosos y fallidos (credenciales incorrectas, campos vacíos, correo duplicado, etc.) para las historias de usuario HU-01 a la HU-06. Con base en los resultados se realizaron ajustes en validaciones y mensajes. En la tabla 4 se presenta la tabla de casos de prueba para la HU-04. En las siguientes figuras, se puede evidenciar la captura de los resultados de los casos de prueba realizados para la HU-04 (registro de usuario). Por último, en el enlace indicado 📁 PRUEBAS Y VALIDACIONES 1 Se encuentran los casos de prueba y las evidencias de los resultados de estos mismos para el Sprint 1.

Tabla 4. Casos de prueba HU-04

Identificación	Nombre	Clase/Método	HU	Entrada	Resultado esperado
CP-HU04-F-01	Correo obligatorio	`RegisterPage` / validación al presionar **Registrarse**	HU-04	Formulario vacío y tap en `Registrarse`	Se muestra el mensaje `Campo obligatorio`.

CP-HU04-F-02	Formato correo inválido	`RegisterPage` / validación del campo correo	HU-04	Correo: `usuario sin arroba`; demás campos sin validar explícitamente; tap en `Registrarse`	Se muestra el mensaje `Correo inválido`.
CP-HU04-F-03	Apodo obligatorio	`RegisterPage` / validación al presionar **Registrarse**	HU-04	Formulario vacío y tap en `Registrarse`	Se muestra el mensaje `Campo obligatorio`.
CP-HU04-F-04	Apodo muy corto	`RegisterPage` / validación del campo apodo	HU-04	Apodo: `ab`, correo: `test@gmail.com`, contraseña: `pass12!`, confirmación: `pass12!`	Se muestra `El apodo debe tener mínimo 3 caracteres`.
CP-HU04-F-05	Contraseña obligatoria	`RegisterPage` / validación del campo contraseña	HU-04	Apodo: `JuanP`, correo: `test@gmail.com`, contraseña vacía; tap en `Registrarse`	Se muestra `Campo obligatorio`.
CP-HU04-F-06	Contraseña mínima	`RegisterPage` / validación de longitud de contraseña	HU-04	Apodo: `JuanP`, correo: `test@gmail.com`, contraseña: `123`, confirmación: `123`	Se muestra un mensaje que contiene `5 caracteres`.
CP-HU04-F-07	Confirmar contraseña no coincide	`RegisterPage` / validación de confirmación de contraseña	HU-04	Apodo: `JuanP`, correo: `test@gmail.com`, contraseña: `pass12!`, confirmación: `pass99!`	Se muestra `Las contraseñas no coinciden`.

CP-HU04-B-01	Campos vacíos retornan 400 campos obligatorios	`userController.registerLocal`	HU-04	`username: ''`, `correo: ''`, `password: ''`, `aceptaTerminos: true`	Respuesta `400` con `{ message: 'Todos los campos son obligatorios' }` y sin consulta a DB.
CP-HU04-B-02	Sin aceptar términos retorna 400	`userController.registerLocal`	HU-04	`username: 'JuanP'`, `correo: 'nuevo@pasto.com'`, `password: 'pass123'`, `aceptaTerminos: false`	Respuesta `400` con `{ message: 'Debes aceptar los términos y condiciones para registrarte' }` y sin consulta a DB.
CP-HU04-B-03	Apodo duplicado retorna 409	`userController.registerLocal`	HU-04	`username: 'JuanP_existente'`, correo y password válidos, términos aceptados	Respuesta `409` con `{ message: 'El apodo ya está en uso' }` y no se ejecuta hash de contraseña.

CP-HU04-B-04	Correo duplicado retorna 409	`userController.registerLocal`	HU-04	`username: 'JuanP'`, `correo: 'nuevo@pasto.com'`, `password: 'pass123'`, `aceptaTerminos: true`; username libre y correo existente en mock	Respuesta `409` con `{ message: 'El correo ya está registrado' }` y no se ejecuta hash de contraseña.
CP-HU04-B-05	Registro exitoso retorna 201 con user y token	`userController.registerLocal`	HU-04	Datos válidos: `username: 'JuanP'`, `correo: 'nuevo@pasto.com'`, `password: 'pass123'`, `aceptaTerminos: true`	Respuesta `201` con objeto `user` y `token`; se genera token con `{ id: 10, rol: 'usuario' }`.
CP-HU04-B-06	Password se hashea con bcrypt y 12 rondas	`userController.registerLocal`	HU-04	Datos válidos con usuario no existente y registro exitoso	`bcrypt.hash` es llamado una vez con `('pass123', 12)`.
CP-HU04-B-07	Nuevo usuario recibe rol usuario y auth_provider local por defecto	`userController.registerLocal`	HU-04	Datos válidos con inserción exitosa	En el insert se envía un objeto con `rol: 'usuario'`, `auth_provider: ['local']` y `estado: 'activo'`.

CP-HU04-B-08	Token se genera con id y rol del usuario creado	`userController.registerLocal`	HU-04	Datos válidos con usuario creado `id: 13`, `rol: 'usuario'`	`generateToken` se llama con `{ id: 13, rol: 'usuario' }` y la respuesta `201` incluye `token: 'auto-login-jwt'`.
CP-HU04-B-09	Error de base de datos retorna 500	`userController.registerLocal`	HU-04	Datos válidos, pero el insert de usuario devuelve `error: new Error('DB connection failed')`	Respuesta `500` con `{ message: 'DB connection failed' }`.
CP-HU04-B-10	Campos obligatorios ausentes retornan 400 sin consultar DB	`userController.registerLocal`	HU-04	`username: 'JuanP'`, `correo: ''`, `password: 'pass123'`, `aceptaTerminos: true`	Respuesta `400` con `{ message: 'Todos los campos son obligatorios' }` y sin consulta a DB.
CP-HU04-I-01	Integral: registro exitoso	`POST /api/auth/register`	HU-04	`username: luna_<timestamp>`, `correo: luna_<timestamp>@mail.com`, `password: 'pass123'`, `aceptaTerminos:`	Respuesta `201`, incluye `token`, `user`, `user.correo` igual al enviado y

				`true`	`user.rol = 'usuario'`.
CP-HU04-I-02	Integral: correo ya registrado	`POST /api/auth/register`	HU-04	Primer registro válido; segundo registro con mismo `correo` y distinto `username`	Respuesta `409` con `{ message: 'El correo ya está registrado' }`.
CP-HU04-I-03	Integral: apodo duplicado	`POST /api/auth/register`	HU-04	Primer registro válido; segundo registro con mismo `username` y distinto `correo`	Respuesta `409` con `{ message: 'El apodo ya está en uso' }`.

Figura 5. Pruebas unitarias en Backend para registro de usuario

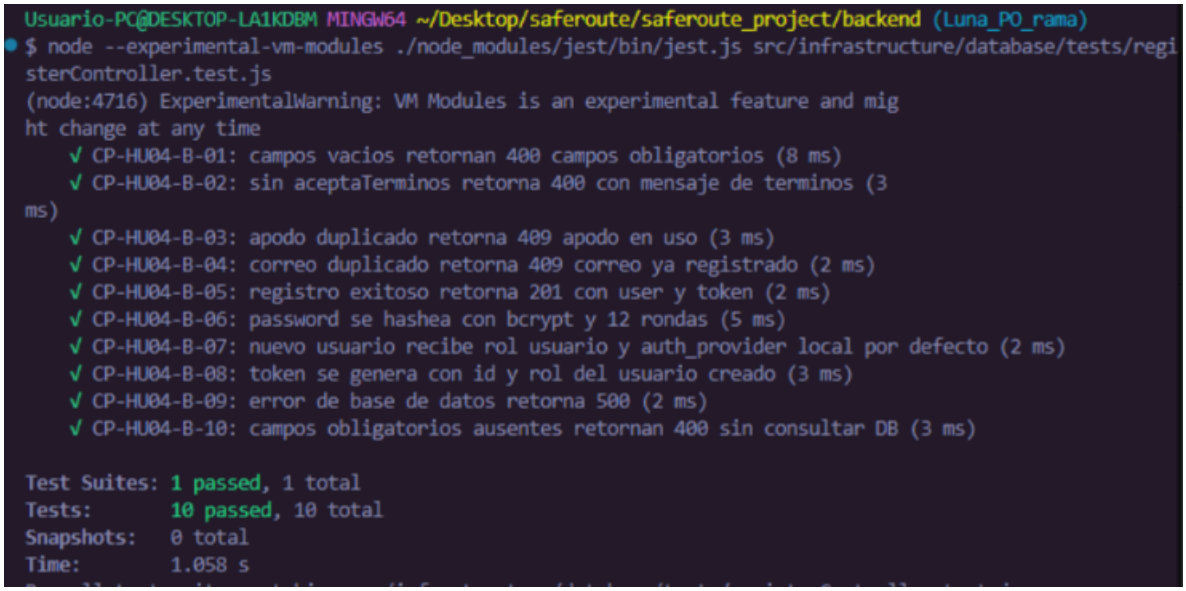


Figura 6. Pruebas unitarias en Frontend-Móvil para registro de usuario (Parte 1)

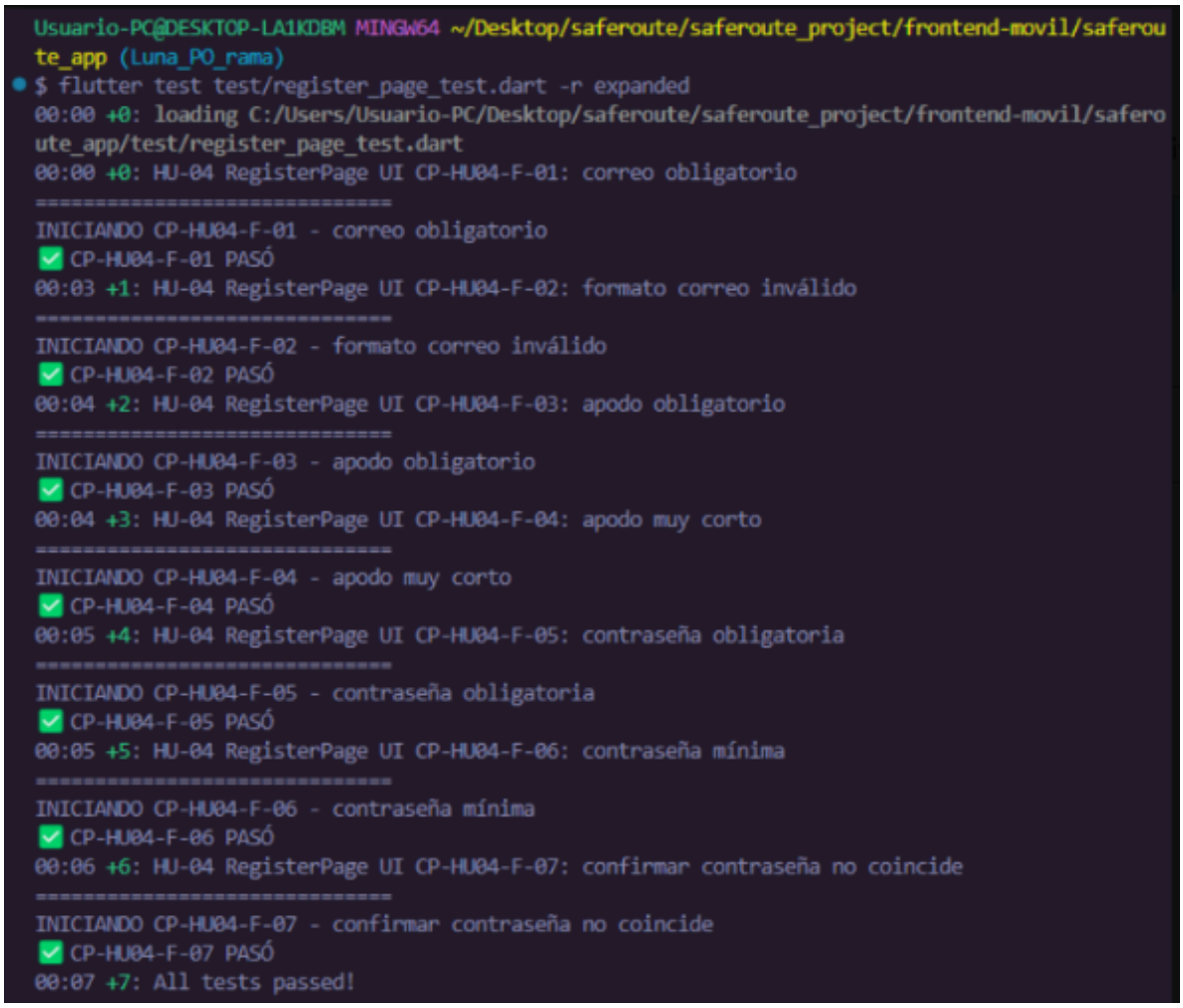
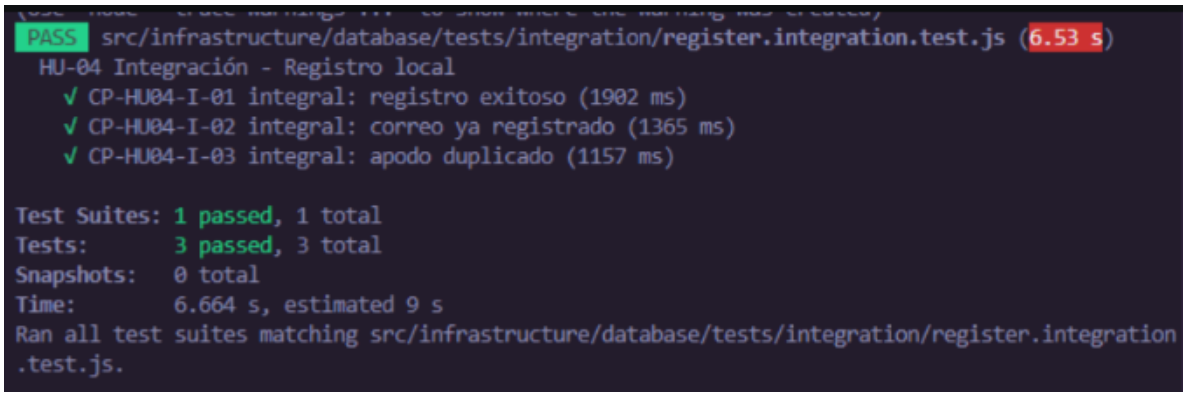


Figura 7. Pruebas Integrales - Registro de usuario



2.1.3. Evaluación

En esta sección, se presentan los resultados obtenidos de realizar los eventos: *Sprint Review y Retrospective.*

- *Sprint Review*

Durante el Sprint se decidió desarrollar un total de 6 historias de usuario incluidas en el Sprint Backlog. De acuerdo con el Definition of Done (DoD) establecido, se logró completar la totalidad de las historias de usuario planificadas, cumpliendo con los criterios definidos para su finalización.

No se presentaron impedimentos significativos que afectarán el desarrollo o la entrega de las funcionalidades propuestas. Sin embargo, se identificó como actividad pendiente la realización de ajustes relacionados con el diseño visual y estilos CSS, los cuales se irán refinando y adaptando progresivamente conforme se avance en el desarrollo de las demás historias de usuario en los próximos sprints.

2.2. Sprint 2

2.2.1. Planeación del *Sprint 2*

Con base en las necesidades identificadas y priorizadas, en el *Sprint Planning Meeting* se seleccionaron las historias de usuario a realizar en el *Sprint*, las cuales se pueden observar en la Tabla 5.

Tabla 5. Sprint backlog

Id	Descripción de la Historia de Usuario	Prioridad	Complejidad
HU7	Como usuario quiero recibir alertas cuando se reporte un hurto cercano a mi ubicación actual para tomar medidas de autocuidado.	56	**Media**
HU8	Como usuario quiero visualizar en un mapa interactivo con los puntos donde se han reportado hurtos para identificar zonas de alto riesgo en tiempo real (actualización automática del mapa en un intervalo máximo de 1 minuto después del registro de un nuevo incidente).	56	**Alta**
HU9	Como usuario quiero poder filtrar los hurtos por fecha, horario y zona para hacer comparaciones simples entre diferentes momentos y lugares y poder restablecer los filtros cuando no los necesite.	49	**Media**

El *Sprint Goal* establecido fue: *Implementar el módulo de mapa interactivo en la aplicación, permitiendo la visualización en tiempo real de reportes de hurto, la aplicación de filtros por fecha, horario y zona, y la configuración de alertas por cercanía, con el fin de apoyar al usuario en la identificación de zonas de riesgo y la toma de decisiones seguras.*

En la Tabla 6, se puede observar el *Definition of Done* (DoF) establecido para este Sprint con las personas responsables por cada tarea.

Tabla 6. Definition of Done del Sprint

Id-HU	Tarea	Responsable	Tiempo estimado (HH:MM)	Tiempo realizado (HH:MM)
HU7	Especificar la HU-07	**Luna**	0:20	0:10
	Diseñar estructuras para configuración de alertas, actualizar diagrama y migracion SQL	**Sarah**	0:45	0:20
	Definir endpoints, lógica de alertas, manejo de permisos y errores	**Fernanda**	4:00	3:30
	Implementar servicio de cálculo de alertas y validar permisos y errores	**Fernanda**	6:00	5:00
	Configurar uso de GPS y permisos	**Fernanda**	4:00	3:00
	Pantalla de configuración de alertas y consumo de endpoint	**Fernanda**	2:00	2:00
	Integrar notificaciones push	**Fernanda**	8:00	6:00
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:10
	Ejecutar casos de prueba	**Luna**	3:00	3:00
HU8	Especificar la HU-08	**Luna**	0:20	0:10
	Crear índices para consultas de mapa, actualizar diagrama	**Sarah**	0:20	0:20
	Definir endpoints para mapa y actualizaciones, GET /mapa/reportes y GET /mapa/nuevos?desde=timestamp	**Fernanda**	6:00	6:00
	Implementar lógica de mapa de calor	**Fernanda**	6:00	4:00
	Integrar SDK y crear la pantalla principal del mapa.	**Sarah**	3:00	2:30
	Consumir el endpoint GET /mapa/reportes y mostrar los hurtos como marcadores interactivos con ventana emergente (tipo, fecha, hora).	**Sarah**	2:00	1:00
	Implementar actualización automática	**Sarah**	2:30	1:30

	del mapa consultando GET /mapa/nuevos y agregando marcadores sin recargar.			
	Implementar el mapa de calor (heatmap) y agregar un botón para alternar entre marcadores y vista de calor.	**Sarah**	3:00	2:00
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:10
	Ejecutar casos de prueba	**Luna**	3:00	3:00
HU9	Especificar la HU-09	**Luna**	0:20	0:10
	Crear índices para filtros, actualizar diagrama	**Sarah**	0:20	0:20
	Definir endpoint de filtros para mapa	**Fernanda**	4:00	4:00
	Implementar lógica de franjas horarias y combinación de filtros	**Fernanda**	4:00	4:00
	Diseño de UI de filtros sobre el mapa	**Sarah**	2:00	1:00
	Integración con el mapa	**Fernanda**	5:00	3:00
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:10
	Ejecutar casos de prueba	**Luna**	3:00	2:00

2.2.2. Ejecución del *Sprint 2*

A continuación, se presenta una síntesis de los resultados obtenidos en la ejecución de las tareas establecidas para el *DoD*.

- **Especificación de la Historia de Usuario**

En el enlace indicado ESPECIFICACIÓN 2, se puede observar la especificación completa de las historias de usuario seleccionadas para el Sprint 2. En la Tabla 7, se presenta a manera de ejemplo la especificación correspondiente a la HU-07, la cual describe el proceso de configuración y recepción de alertas de hurtos cercanos, incluyendo sus reglas de funcionamiento y criterios de aceptación.

Tabla 7. Ejemplo de la especificación de la HU-07

Historia de Usuario

Código:	HU-07		
Nombre:	**Recibir alertas de incidentes de hurto cercanos**		
Actor:	**Usuario / Ciudadano**		
Descripción	Como usuario quiero recibir alertas cuando se reporte un hurto cercano a mi ubicación actual para tomar medidas de autocuidado.		
Criterios de aceptación:	**CID**	**Condición**	**Resultado**
	1	El usuario tiene activado el servicio de ubicación (GPS o geolocalización del dispositivo) y otorga permiso a la aplicación para acceder a su ubicación.	El sistema obtiene la ubicación actual del usuario y permite configurar un radio de alerta (ej. 200 m, 500 m o 1 km).
	2	El usuario intenta activar las alertas sin tener habilitado el servicio de ubicación del dispositivo.	El sistema solicita activar el GPS y muestra un mensaje informando que la funcionalidad de alertas requiere acceso a la ubicación.
	3	Se registra un hurto cuya ubicación se encuentra dentro del radio configurado por el usuario y las notificaciones están activas.	El usuario recibe una notificación push con información básica del incidente (tipo de hurto, hora y zona general).
	4	El usuario desactiva las notificaciones o revoca el permiso de ubicación.	El sistema deja de enviar alertas al dispositivo del usuario.
	5	Incidente fuera del radio configurado	No se envía notificación
	6	Múltiples incidentes cercanos en corto tiempo	El sistema gestiona notificaciones sin duplicidad

- **Diseño e implementación de la base de datos**

Durante el Sprint 2 se diseñaron e implementaron las estructuras necesarias para soportar las funcionalidades de **mapa, filtros** y **alertas del sistema**. En este Sprint se creó la estructura de configuración de alertas para los usuarios, permitiendo almacenar parámetros como el radio de notificación y el estado de activación. Adicionalmente, se realizaron ajustes al modelo de datos mediante la actualización del diagrama de clases/ER y la elaboración de las migraciones SQL correspondientes, esto con el fin de mejorar el rendimiento del aplicativo, se implementaron índices para optimizar las consultas del mapa, así como índices orientados a los filtros por **fecha, horario** y **zona**. En la **Figura 8 y 9** se

presenta la tabla para las alertas del sistema y la migración SQL respectivamente. En el enlace indicado BASE DE DATOS 2, se puede consultar el diagrama actualizado y el código completo de las estructuras implementadas durante el Sprint 2.

Figura 8. Tabla alertas

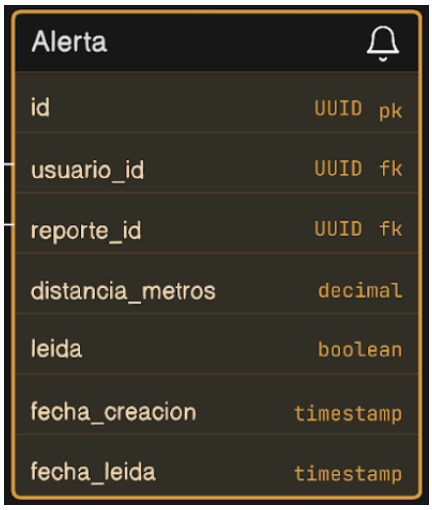


Figura 9. Migracion SQL para alertas

```sql
-- TABLA: alertas
CREATE TABLE IF NOT EXISTS public.alertas (
    id               UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    usuario_id       UUID        NOT NULL,
    reporte_id       UUID        NOT NULL,
    distancia_metros NUMERIC(8,2),
    leida            BOOLEAN     NOT NULL DEFAULT false,
    fecha_creacion   TIMESTAMP   NOT NULL DEFAULT now(),
    fecha_leida      TIMESTAMP,

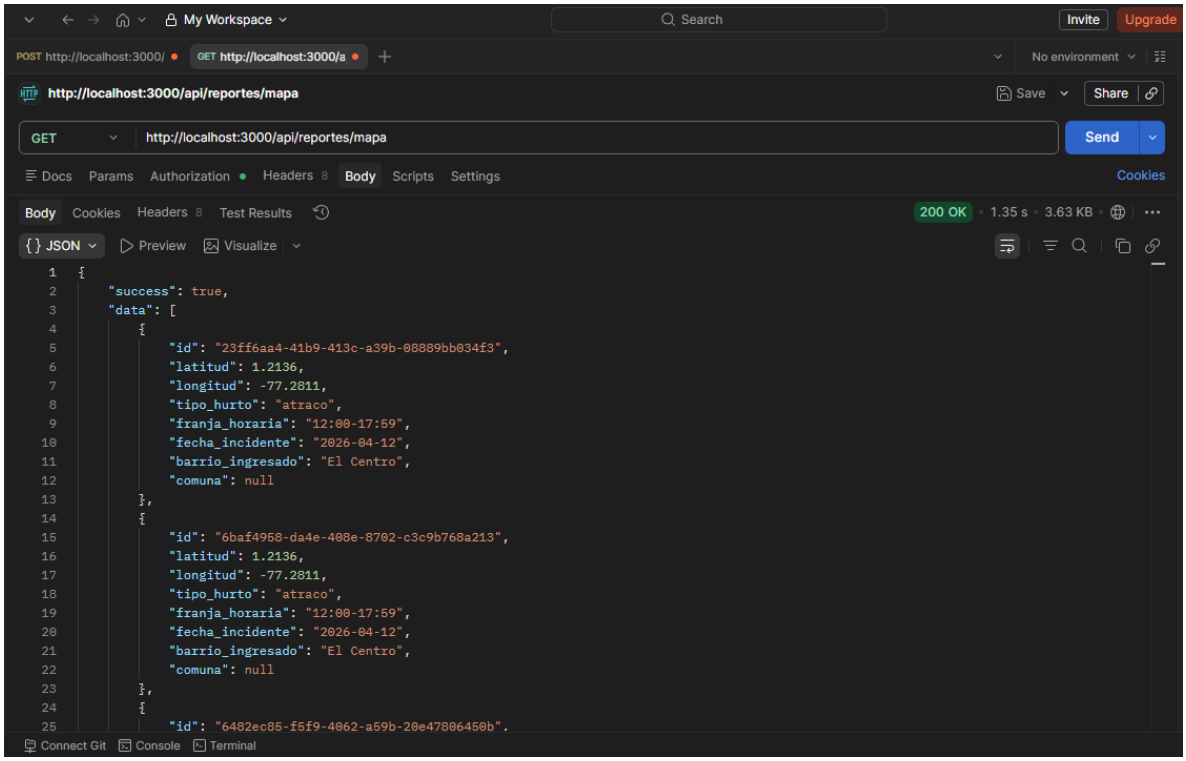
    CONSTRAINT fk_alerta_usuario
        FOREIGN KEY (usuario_id) REFERENCES public.usuarios(id) ON DELETE CASCADE,
    CONSTRAINT fk_alerta_reporte
        FOREIGN KEY (reporte_id) REFERENCES public.reportes(id) ON DELETE CASCADE
);
```

- **Desarrollo del Back-end**

Durante el Sprint 2 se definieron e implementaron los endpoints correspondientes a las funcionalidades de **mapa, filtros y alertas**, permitiendo la consulta de reportes georreferenciados y su actualización automática en tiempo real, por esto se desarrollaron los servicios necesarios para el cálculo de alertas cercanas, incorporando validaciones relacionadas con permisos de ubicación y manejo de errores. Adicionalmente, se implementaron los endpoints para soportar la carga inicial del mapa y la consulta incremental de nuevos reportes, así como la lógica requerida para la generación del mapa

de calor. También se incluyó el endpoint de filtros, integrando el procesamiento de franjas horarias y combinaciones de filtros por fecha, horario y zona. En la **Figura 10** se presenta la evidencia de funcionamiento del endpoint **GET /api/reportes/mapa**. En el enlace indicado BACKEND 2 se encuentran las demás evidencias de las pruebas realizadas a los endpoints desarrollados durante el Sprint 2.

Figura 10. Prueba del endpoint GET /api/reportes/mapa

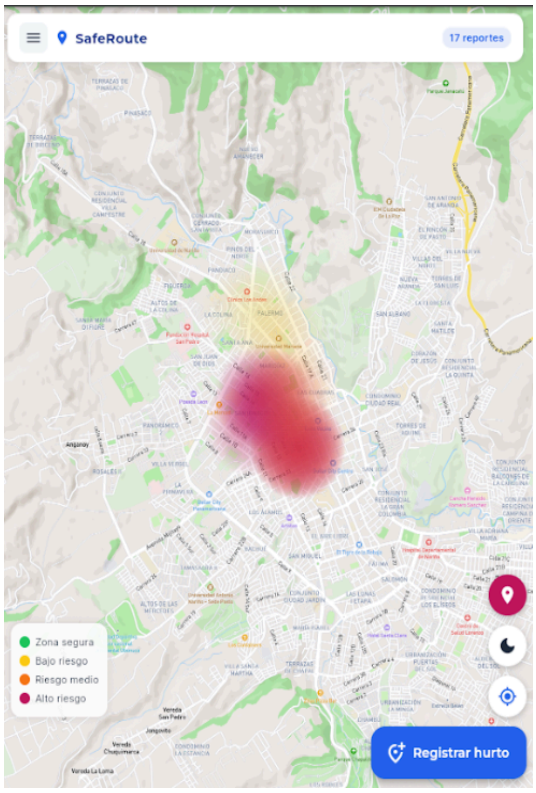


- **Diseño de la Interfaz**
  - **Desarrollo del Front-end Flutter (Usuario)**

Durante el Sprint 2 se implementaron las interfaces móviles relacionadas con las funcionalidades de **mapa, filtros y alertas**. En este Sprint se configuró el uso de GPS y permisos de ubicación, permitiendo al usuario activar la funcionalidad de alertas cercanas. Asimismo, se desarrolló la pantalla de configuración de alertas y se integró el consumo de los endpoints definidos en el backend. Además , se integró el SDK de mapas para construir la pantalla principal, donde se muestran los reportes de hurto mediante marcadores interactivos, incluyendo ventanas emergentes con información del incidente. También se implementó la actualización automática del mapa consultando el endpoint de nuevos reportes, y se incorporó la funcionalidad de mapa de calor, permitiendo alternar entre la vista de marcadores y la capa de calor. Finalmente, se diseñó e integró la interfaz de filtros sobre el mapa para consultar información por fecha, horario y zona. En la **Figura 11** se presenta la **implementación del mapa de calor** que hace parte de la HU-08. En el enlace

indicado FRONTEND MOVIL 2 se encuentran las demás pantallas desarrolladas durante el Sprint 2.

Figura 11. Implementación del Mapa de calor



- **Pruebas y Validación**

Durante el Sprint 2 se diseñaron y ejecutaron escenarios de prueba para las funcionalidades implementadas en las historias de usuario **HU-07, HU-08 y HU-09**, validando tanto casos exitosos como fallidos relacionados con permisos de ubicación, visualización de reportes en el mapa, actualización automática y aplicación de filtros por fecha, horario y zona. Con base en los resultados obtenidos, se realizaron ajustes en validaciones, mensajes y comportamiento del sistema para garantizar una correcta respuesta ante diferentes condiciones de uso. En la **Tabla 8** se presentan los casos de prueba correspondientes a la **HU-09**, los cuales permiten evidenciar la correcta aplicación y restablecimiento de filtros. En la **Figura 12** se presenta una evidencia de la ejecución de los casos de prueba asociados a dicha historia de usuario. Finalmente, en el enlace indicado PRUEBAS Y VALIDACIONES 2, se encuentran disponibles los casos de prueba completos y las evidencias de los resultados obtenidos durante el Sprint 2.

Tabla 8. Casos de prueba HU-09

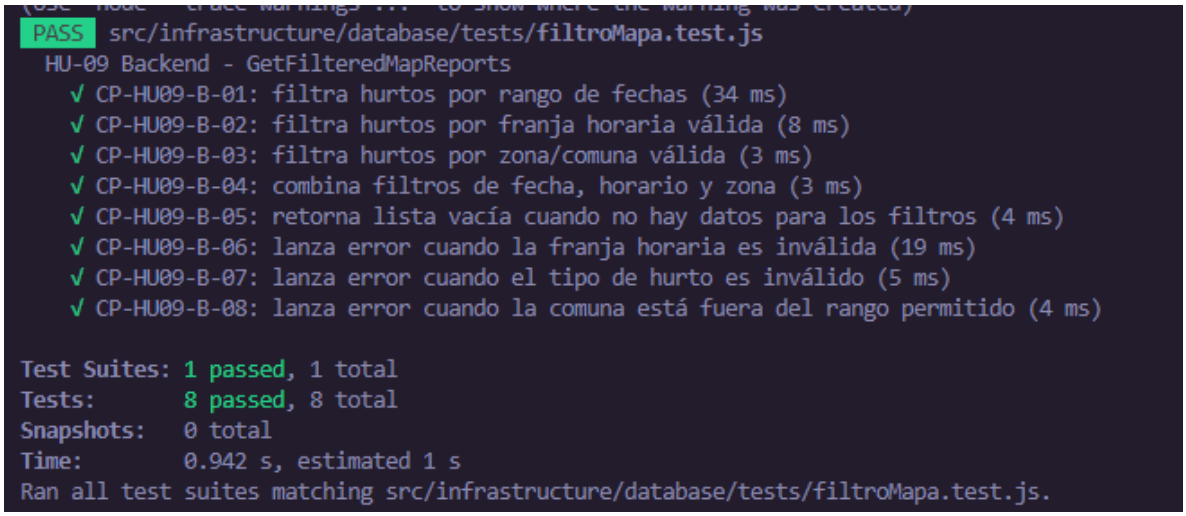
Identificación	Nombre	Clase/Método	HU	Entrada	Resultado esperado
CP-HU09-F-01	Muestra drawer de filtros con secciones	FilterDrawer / render del drawer	HU-09	Abrir el FilterDrawer desde el botón Abrir	Se muestra el título Filtros y las secciones Comuna, Rango horario, Tipo de hurto y Fecha del incidente.
CP-HU09-F-02	Permite seleccionar una comuna	FilterDrawer / selección de comuna	HU-09	Abrir el drawer, hacer scroll hasta el chip 3 y seleccionarlo	La interacción se realiza correctamente y el botón Aplicar filtros permanece visible.
CP-HU09-F-03	Permite seleccionar una franja horaria	FilterDrawer / selección de franja	HU-09	Abrir el drawer, hacer scroll hasta 06:00-11:59 y seleccionarlo	La selección se realiza correctamente y el botón Aplicar filtros permanece visible.
CP-HU09-F-04	Permite seleccionar tipo de hurto	FilterDrawer / selección de tipo de hurto	HU-09	Abrir el drawer, hacer scroll hasta	La selección se realiza correctamente y el botón Aplicar

				Atraco y seleccionarlo	filtros permanece visible.
CP-HU09-F-05	Permite restablecer filtros desde el drawer	FilterDrawer / restablecimiento de filtros	HU-09	Abrir el drawer, seleccionar una comuna y pulsar Restablecer filtros	El botón Restablecer filtros funciona sin errores y el drawer mantiene visible Aplicar filtros.
CP-HU09-F-06	Botón aplicar filtros cierra el drawer	FilterDrawer / acción aplicar	HU-09	Abrir el drawer y pulsar Aplicar filtros	El drawer se cierra y el encabezado Filtros deja de ser visible.
CP-HU09-B-01	Filtra hurtos por rango de fechas	GetFilteredMapReports.execute()	HU-09	fechaDesde: '2026-04-01', fechaHasta: '2026-04-10'	El repositorio es llamado con esos filtros y el caso de uso retorna la lista filtrada esperada.
CP-HU09-B-02	Filtra hurtos por franja horaria válida	GetFilteredMapReports.execute()	HU-09	franjas: ['06:00-11:59']	El repositorio es llamado con la franja indicada y el caso de uso retorna la lista correspondiente.

CP-HU09-B-03	Filtra hurtos por zona/comuna válida	GetFilteredMapReports.execute()	HU-09	comunas: [5]	El repositorio es llamado con comunas: [5] y retorna la lista filtrada esperada.
CP-HU09-B-04	Combina filtros de fecha, horario y zona	GetFilteredMapReports.execute()	HU-09	comunas: [4], franjas: ['18:00-23:59'], fechaDesde: '2026-04-01', fechaHasta: '2026-04-30'	El repositorio recibe todos los filtros combinados y retorna la lista filtrada correspondiente.
CP-HU09-B-05	Retorna lista vacía cuando no hay datos para los filtros	GetFilteredMapReports.execute()	HU-09	comunas: [9], franjas: ['00:00-05:59'], fechaDesde: '2026-04-01', fechaHasta: '2026-04-02'	El repositorio es llamado con esos filtros y el resultado es [].

CP-HU09-B-06	Lanza error cuando la franja horaria es inválida	GetFilteredMapReports.execute()	HU-09	franjas: ['madrugada']	El caso de uso lanza un error que coincide con Franjas inválidas y no llama al repositorio.
CP-HU09-B-07	Lanza error cuando el tipo de hurto es inválido	GetFilteredMapReports.execute()	HU-09	tipos: ['robo']	El caso de uso lanza un error que coincide con Tipos de hurto inválidos y no llama al repositorio.
CP-HU09-B-08	Lanza error cuando la comuna está fuera del rango permitido	GetFilteredMapReports.execute()	HU-09	comunas: [0, 13]	El caso de uso lanza un error que coincide con Comunas inválidas y no llama al repositorio.

Figura 12. Evidencia de ejecución de pruebas HU-09 en Backend



2.2.3. Evaluación

En esta sección, se presentan los resultados obtenidos de realizar los eventos: *Sprint Review y Retrospective.*

- *Sprint Review*

En el Sprint 2 se desarrollaron un total de *3* historias de usuario planificadas en el Sprint Backlog, correspondientes a las funcionalidades de mapa, filtros y alertas (HU-07, HU-08 y HU-09). De acuerdo con el Definition of Done (DoD) establecido, se logró completar la totalidad de las historias de usuario previstas, cumpliendo con los criterios de aceptación definidos para su finalización.

Durante la revisión, los stakeholders destacaron que el avance del proyecto es significativo y se evidencia un progreso positivo en la construcción de las funcionalidades principales del sistema, adicionalmente, se sugirió fortalecer las validaciones asociadas al módulo de alertas para garantizar mayor coherencia en las notificaciones emitidas, implementar un mecanismo que permita identificar y unificar posibles reportes duplicados de un mismo incidente (por ejemplo, registros realizados por víctimas y testigos sobre el mismo hecho), e incorporar mensajes informativos en los filtros por fecha para orientar al usuario sobre rangos válidos y mejorar la experiencia de uso.

No se presentaron impedimentos significativos que afectaran el desarrollo o la entrega de las funcionalidades propuestas. Sin embargo, se determinó como actividad pendiente continuar con mejoras relacionadas con el diseño visual y experiencia de usuario, especialmente en la presentación de la información en pantalla.

Finalmente, como parte de las decisiones tomadas para el siguiente Sprint, se incorporó una nueva historia de usuario al Product Backlog:

HU-18 – Recuperación de contraseña por correo electrónico:

Como usuario/administrador, quiero recuperar mi contraseña mediante el envío de un enlace de restablecimiento a mi correo electrónico, para poder acceder nuevamente al sistema en caso de olvido.

- *Sprint Retrospective*

En la retrospectiva del Sprint 2 se identificaron oportunidades de mejora relacionadas con la estimación de tareas y la organización del trabajo. Se evidenció que algunas actividades simples fueron sobreestimadas, mientras que tareas técnicas complejas requirieron mayor esfuerzo del previsto, por esta razón, el equipo propuso aplicar técnicas de estimación y realizar seguimiento en retrospectivas futuras sobre las desviaciones presentadas.

Se concluyó que es necesario fortalecer el conocimiento técnico del equipo mediante más espacios de investigación y sesiones internas de transferencia de conocimientos, esto con el fin de reducir bloqueos en el desarrollo y mejorar la eficiencia del Sprint.

2.3. Sprint 3

2.3.1. Planeación del *Sprint 3*

Con base en las necesidades identificadas y priorizadas, en el *Sprint Planning Meeting* se seleccionaron las historias de usuario a realizar en el *Sprint*, las cuales se pueden observar en la Tabla 9.

Tabla 9. Sprint backlog

Id	Descripción de la Historia de Usuario	Prioridad	Complejidad
HU10	Como administrador quiero ver un panel con el listado de incidentes de hurto registrados, filtrables por fecha, zona, para gestionar y validar la información del sistema.	42	**Alta**
HU11	Como usuario quiero ver estadísticas básicas (cantidad de hurtos por periodo y su tendencia) calculadas a partir de reportes verificados, para comparar si los casos están aumentando o disminuyendo en mi zona.	42	**Alta**
HU12	Como usuario quiero que la app me muestre un listado de los barrios o sectores con mayor número de hurtos registrados, para identificar cuáles	40	**Media**

	presentan mayor nivel de riesgo al transitar.		
HU13	Como usuario/administrador, quiero recuperar mi contraseña mediante el envío de un enlace de restablecimiento a mi correo electrónico, para poder acceder nuevamente al sistema en caso de olvido.	36	**Baja**

El *Sprint Goal* establecido fue: *Implementar funcionalidades de análisis y administración en CivicTrackIO mediante el desarrollo del panel web para administradores con listado y filtrado de reportes, la incorporación de estadísticas básicas y el ranking de zonas con mayor número de hurtos en la aplicación móvil, además de integrar la opción de recuperación de contraseña por correo electrónico, con el fin de mejorar la toma de decisiones, la visualización de información y el acceso seguro al sistema.*

En la Tabla 10, se puede observar el *Definition of Done* (DoF) establecido para este Sprint con las personas responsables por cada tarea.

Tabla 10. Definition of Done del Sprint

Id-HU	Tarea	Responsable	Tiempo estimado (HH:MM)	Tiempo realizado (HH:MM)
	Mejoras de Sprint 2	**Sarah**	1:00	1:00
	Actualizar documentación ER.	**Sarah**	1:00	1:00
HU10	Especificar la HU-10	**Luna**	0:20	0:10
	Verificar y ajustar índices existentes para consultas del panel admin (fecha/zona)	**Sarah**	1:30	1:00
	Definir endpoint de listado, detalle y resumen	**Fernanda**	3:00	2:00
	Implementar lógica de filtrado y paginación; y proteger rutas del panel	**Fernanda**	4:00	3:00
	Diseñar pantalla principal de panel de admin (Resumen e incidentes)	**Sarah**	2:00	7:00
	Implementar consumo de API y filtrado	**Fernanda**	4:00	3:00
	Implementar vista de detalle ampliado	**Sarah**	2:00	7:00
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:20
	Ejecutar casos de prueba	**Luna**	3:00	3:00

HU12	Especificar la HU-11	**Luna**	0:20	0:10
	Definir consultas y crear vista vw_estadisticas_basicas	**Sarah**	1:30	1:00
	Soporte para comparación de periodos	**Sarah**	1:30	1:00
	Definir endpoints de estadísticas	**Fernanda**	3:00	3:00
	Implementar consulta de estadísticas por periodo y comparación.	**Fernanda**	4:00	3:00
	Diseñar pantalla de “Estadísticas”	**Sarah**	1:00	0:45
	Implementar comparación de periodos	**Sarah**	1:00	0:45
	Integrar API de estadísticas	**Fernanda**	4:00	4:00
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:20
	Ejecutar casos de prueba	**Luna**	3:00	3:00
HU12	Especificar la HU-12	**Luna**	0:20	0:10
	Definir consulta para ranking de comunas/sectores con más hurtos	**Sarah**	0:45	1:00
	Crear vista vw_top_zonas_hurtos (conteo por comuna/sector)	**Sarah**	1:30	1:00
	Verificar y optimizar índices por zona y fecha para ranking	**Sarah**	0:45	1:00
	Implementar lógica para obtener top N zonas (ordenado por cantidad de hurtos)	**Fernanda**	6:00	4:00
	Agregar filtros opcionales por rango de fechas	**Fernanda**	3:00	3:00
	Implementar paginación o límite de resultados (ej. top 5, top 10)	**Fernanda**	3:00	2:00
	Diseñar pantalla “Zonas más peligrosas” con listado de comunas/sectores	**Sarah**	1:00	0:45
	Consumir endpoint y mostrar ranking con cantidad de hurtos	**Fernanda**	4:00	3:30
	Permitir ordenar/filtrar por periodo (día, semana, mes) (opcional)	**Sarah**	1:00	0:45
	Identificar escenarios de prueba	**Luna**	1:00	0:10

	Diseñar y crear casos de prueba	**Luna**	2:00	0:20
	Ejecutar casos de prueba	**Luna**	3:00	3:00
HU13	Especificar la HU-13	**Luna**	0:20	0:10
	Crear tabla password_resets (usuario_id, token, expiracion, usado, fecha_creacion).y definir relación FK con tabla usuarios.	**Sarah**	1:30	1:00
	Crear índice por token para búsqueda rápida	**Sarah**	0:45	1:00
	Definir endpoint POST /auth/forgot-password (validación de correo registrado y respuesta estándar).	**Fernanda**	5:00	3:00
	Implementar lógica para generar token seguro y guardar en la tabla password_resets con expiración.	**Fernanda**	4:00	3:00
	Implementar servicio de envío de correo con enlace o código de recuperación (SMTP / SendGrid / Supabase Auth).	**Fernanda**	3:00	7:00
	Definir endpoint POST /auth/reset-password para validar token, verificar expiración y cambiar contraseña.	**Fernanda**	3:00	3:00
	Implementar actualización de contraseña con bcrypt y marcar token cómo usado.	**Fernanda**	3:00	3:00
	Implementar manejo de errores y respuestas (token inválido, expirado, usado, correo no existente). y proteger el endpoint para evitar abusos	**Fernanda**	4:00	3:30
	Agregar opción “¿Olvidaste tu contraseña?” en login del panel admin	**Sarah**	1:00	2:30
	Diseñar e implementar formulario para ingresar correo electrónico.	**Sarah**	1:00	2:30
	Consumir endpoint POST /auth/forgot-password y mostrar mensaje de confirmación.	**Fernanda**	2:00	2:00
	Implementar pantalla/formulario de restablecer contraseña (nueva contraseña + confirmación).	**Sarah**	1:00	2:30

	Consumir endpoint POST /auth/reset-password y redirigir al login admin. Validar campos (correo válido, contraseñas iguales, longitud mínima).	**Fernanda**	4:00	3:30
	Agregar opción “¿Olvidaste tu contraseña?” en la pantalla de inicio de sesión.	**Sarah**	1:00	2:30
	Diseñar pantalla “Recuperar contraseña” para ingresar correo.	**Sarah**	1:00	2:30
	Consumir endpoint POST /auth/forgot-password y mostrar confirmación al usuario.	**Fernanda**	3:00	3:00
	Diseñar pantalla “Restablecer contraseña” (nueva contraseña + confirmación).	**Sarah**	1:00	2:30
	Consumir endpoint POST /auth/reset-password y redirigir al login. Implementar validaciones de seguridad básicas (contraseña mínima, confirmación correcta).	**Fernanda**	5:00	2:00
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:20
	Ejecutar casos de prueba	**Luna**	3:00	2:00

2.3.2. Ejecución del *Sprint 3*

A continuación, se presenta una síntesis de los resultados obtenidos en la ejecución de las tareas establecidas para el *DoD*.

- **Especificación de la Historia de Usuario**

En el enlace indicado ESPECIFICACIÓN 3, se puede observar la especificación completa de las historias de usuario seleccionadas para el Sprint 3, correspondientes a funcionalidades de panel administrativo, análisis estadístico y recuperación de acceso. En la Tabla 11, se presenta a manera de ejemplo la especificación correspondiente a la HU-10, la cual describe el proceso de visualización y filtrado de reportes en el panel administrativo, incluyendo sus reglas de negocio y criterios de aceptación.

Tabla 11. Ejemplo de la especificación de la HU-10

Historia de Usuario			
Código:	**HU-10**		
Nombre:	**Visualizar panel de reportes de hurtos**		
Actor:	**Administrador**		
Descripción	Como administrador quiero ver un panel con el listado de incidentes de hurto registrados, filtrables por fecha, zona, para gestionar y validar la información del sistema.		
Criterios de aceptación:	**CID**	**Condición**	**Resultado**
	1	El administrador accede a la sección “Reportes de hurtos” en el panel.	El sistema muestra una tabla con los reportes (ID, apodo, tipo de hurto, fecha, hora, zona).
	2	El administrador aplica filtros por rango de fechas y zona.	El sistema actualiza la tabla mostrando únicamente los reportes que cumplen los filtros.
	3	El administrador selecciona un reporte de la lista.	El sistema muestra el detalle completo del incidente (incluyendo descripción y coordenadas) en una vista ampliada o ventana emergente.

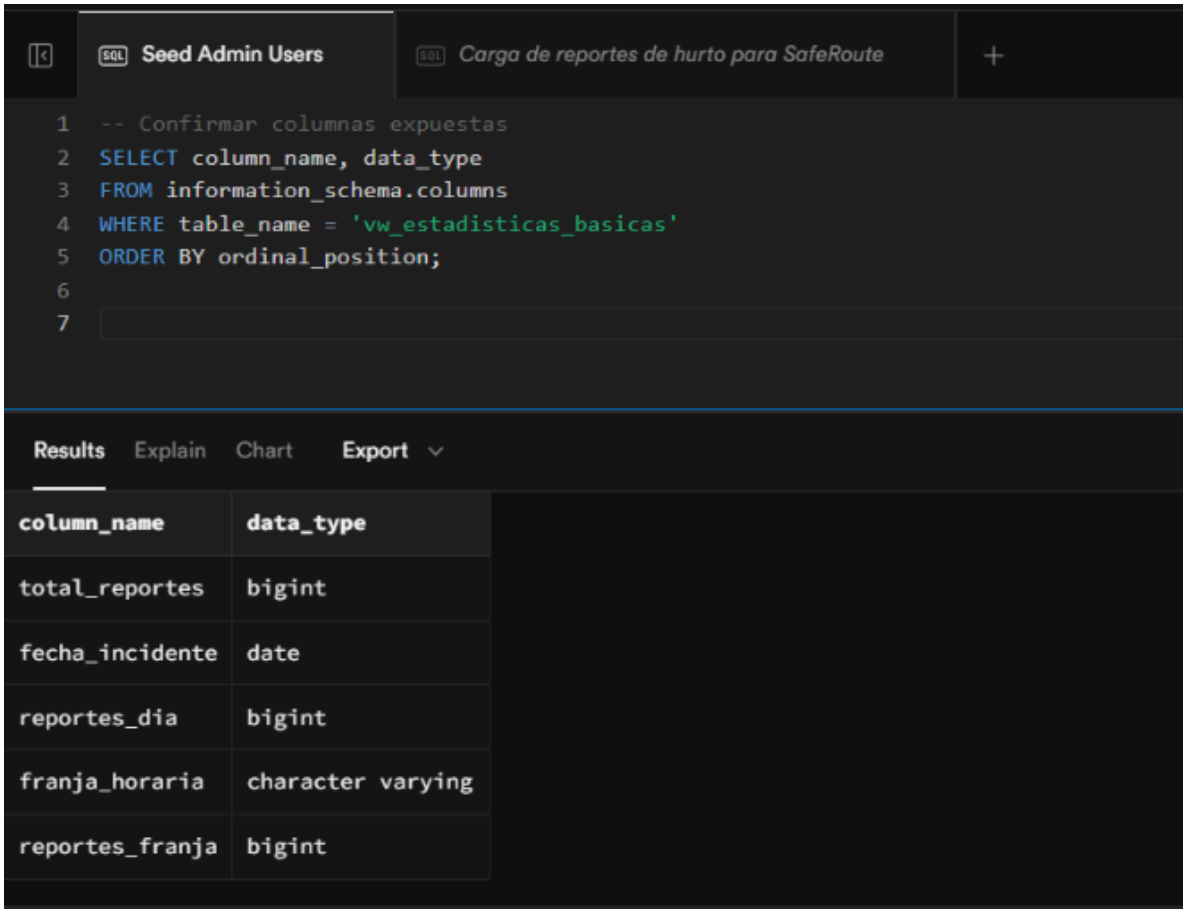
- **Diseño e implementación de la base de datos**

Durante el Sprint 3 se diseñaron e implementaron nuevas estructuras de base de datos orientadas al análisis y visualización de información, así como al soporte del panel administrativo y la recuperación de contraseña. En este Sprint se crearon vistas para el cálculo de estadísticas básicas, logrando consolidar información sobre reportes por periodo y facilitar comparaciones de tendencias, asimismo, se implementó la vista correspondiente al ranking de zonas con mayor número de hurtos, optimizando las consultas necesarias para la funcionalidad de “zonas más peligrosas”.

De igual forma, se verificaron y ajustaron índices existentes relacionados con fecha y zona, con el fin de mejorar el rendimiento de las consultas realizadas por el panel administrativo y para la HU-13 se incorporó la estructura necesaria para el proceso de recuperación de contraseña mediante correo electrónico, creando una tabla para almacenar tokens temporales y su fecha de expiración, asegurando trazabilidad y control del proceso.

En la **Figura 13** se presenta la evidencia de las columnas definidas en la vista de estadísticas básicas. En el enlace indicado 📸 **BASE DE DATOS 3**, se puede consultar el diagrama actualizado y el código completo de las vistas, índices y estructuras implementadas durante el Sprint 3.

Figura 13. Columnas expuestas en la vista vw_estadisticas_basicas



- **Desarrollo del Back-end**

Durante el Sprint 3 se definieron e implementaron endpoints orientados a la consulta y análisis de información estadística, así como a la administración de reportes dentro del panel web. En este Sprint se desarrollaron endpoints para el cálculo de estadísticas básicas por periodo, incluyendo la posibilidad de comparar rangos de fechas, que permiten al usuario analizar tendencias en los reportes registrados. También se implementó la lógica para obtener el ranking de zonas con mayor número de hurtos, aplicando filtros por rango de fechas y límites de resultados (top N).

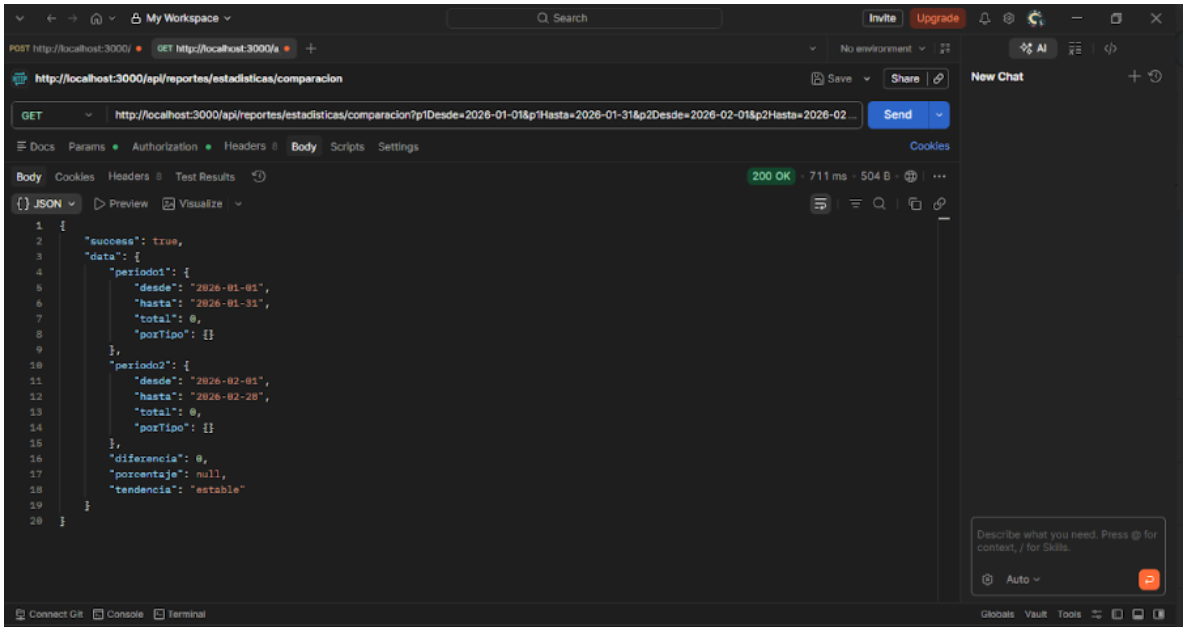
Incluso, se fortaleció el backend administrativo con endpoints protegidos mediante autenticación y control de roles, permitiendo al administrador consultar reportes con filtros, paginación y acceso a detalles ampliados.

Por último, se desarrolló el módulo de recuperación de contraseña (HU-13), implementando los endpoints correspondientes para la generación de tokens seguros, validación de expiración, actualización de contraseña con hash seguro y control de errores

ante intentos inválidos o repetidos, integrando este proceso con un servicio de envío de correo electrónico para permitir al usuario restablecer su acceso al sistema.

En la **Figura 14** se presenta la evidencia del funcionamiento del endpoint de comparación de periodos correspondiente a estadísticas. En el enlace indicado BACKEND 3, se encuentran las demás evidencias de pruebas realizadas a los endpoints desarrollados durante el Sprint 3.

Figura 14. Prueba del endpoint GET /api/reportes/estadisticas/comparacion



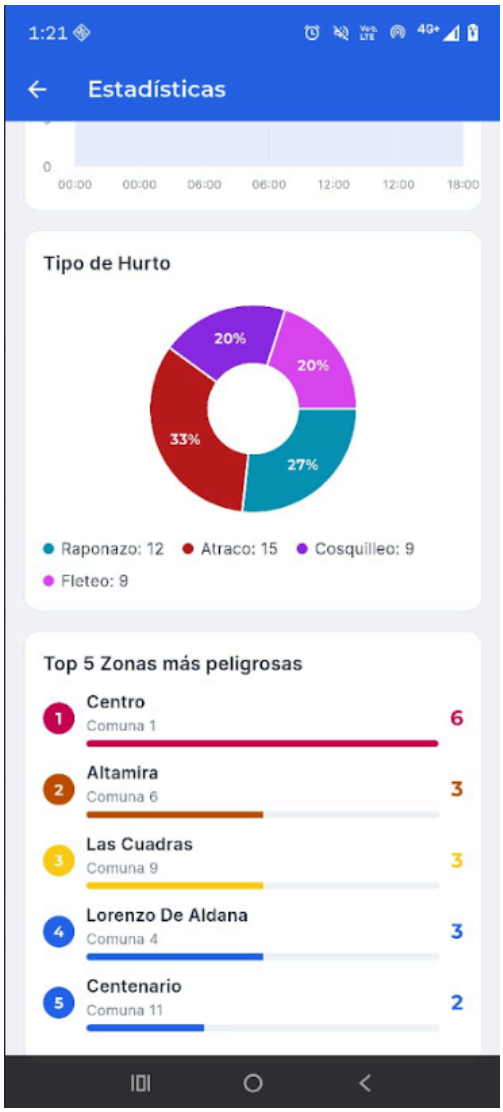
- **Diseño de la Interfaz**
  - *Desarrollo del Front-end Flutter (Usuario)*

Durante el Sprint 3 se implementaron las interfaces móviles correspondientes a las funcionalidades de análisis y visualización de datos. En este Sprint se desarrolló la pantalla de estadísticas básicas (HU-11), lo que permite al usuario consultar reportes agrupados por periodo y visualizar tendencias de aumento o disminución. También se implementó la funcionalidad de ranking de zonas más peligrosas (HU-12), mostrando un listado de sectores con mayor número de reportes y permitiendo realizar filtros por rango de fechas para recalcular los resultados.

Adicionalmente, se diseñaron las interfaces correspondientes al proceso de recuperación de contraseña (HU-13), incorporando pantallas para solicitar el envío del enlace de recuperación y para registrar la nueva contraseña con validaciones básicas de seguridad, estas pantallas consumen los endpoints del backend y presentan mensajes informativos de confirmación o error según el caso.

En la **Figura 15** se presenta la evidencia de las pruebas unitarias realizadas en Flutter para la HU-12 (zonas con mayor número de hurtos). En el enlace indicado FRONTEND MOVIL 3, se encuentran las demás pantallas desarrolladas durante el Sprint 3.

Figura 15. Pantalla "HU-12 Zonas con mayor número de hurtos" (Flutter)



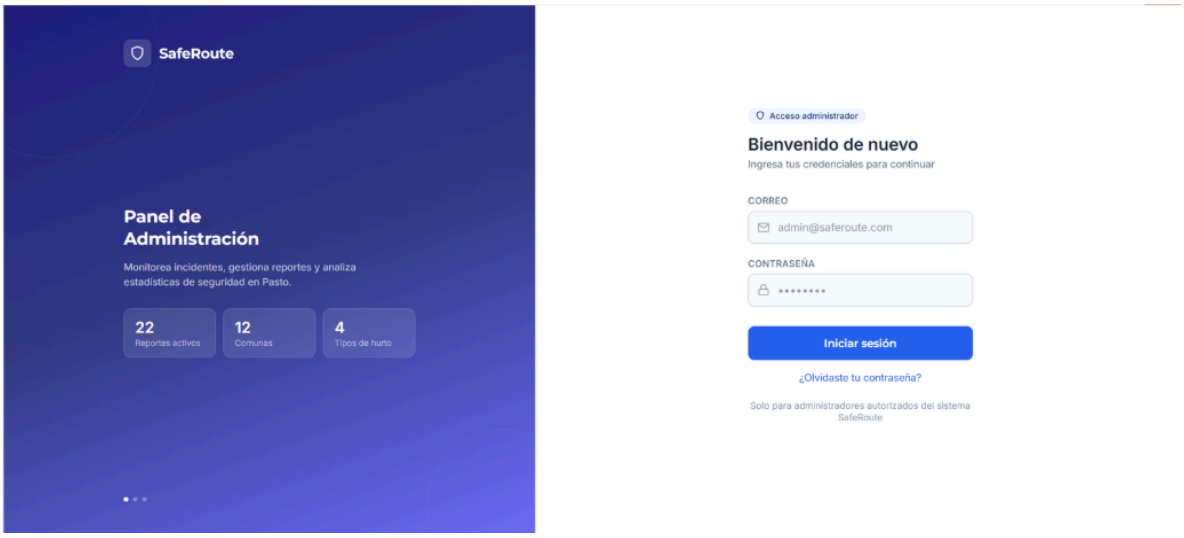
- *Desarrollo del Front-end React (Administrador)*

Durante el Sprint 3 se implementaron las interfaces web del panel administrativo (HU-10), permitiendo al administrador visualizar un listado de incidentes registrados con filtros por fecha y zona, junto con paginación para mejorar la navegación en grandes volúmenes de datos, también se desarrolló una vista de detalle ampliado para consultar información completa de cada reporte.

Además, se integró la interfaz correspondiente a la recuperación de contraseña en el login del panel administrativo (HU-13), incluyendo la opción "¿Olvidaste tu contraseña?" y el formulario de restablecimiento, asegurando que el administrador también pueda recuperar el acceso mediante correo electrónico.

En la **Figura 16** se presentará la evidencia correspondiente a la pantalla de recuperación de contraseña del panel administrativo. En el enlace indicado 🖿 FRONTEND WEB 3, se encuentran las demás interfaces implementadas durante el Sprint 3.

Figura 16. Pantalla de recuperación de contraseña – Panel administrativo (React)



- **Pruebas y Validación**

Durante el Sprint 3 se diseñaron y ejecutaron escenarios de prueba para las funcionalidades implementadas en las historias de usuario HU-10, HU-11, HU-12 y HU-13. Se validaron casos exitosos y fallidos relacionados con la consulta de reportes administrativos, paginación y filtros del panel web, así como la correcta generación de estadísticas básicas, ranking de zonas con mayor riesgo y funcionamiento del proceso de recuperación de contraseña.

En la **Tabla 12** se presentan los casos de prueba correspondientes a la HU-10. En la Figura 17 se evidencia la ejecución exitosa de las pruebas unitarias realizadas para esta historia de usuario en la parte del Frontend-móvil. Finalmente, en el enlace indicado 🖿 PRUEBAS Y VALIDACIONES 3, se encuentran disponibles los casos de prueba completos y evidencias del Sprint 3.

Figura 17. Evidencia de ejecución de pruebas unitarias en React – HU-10

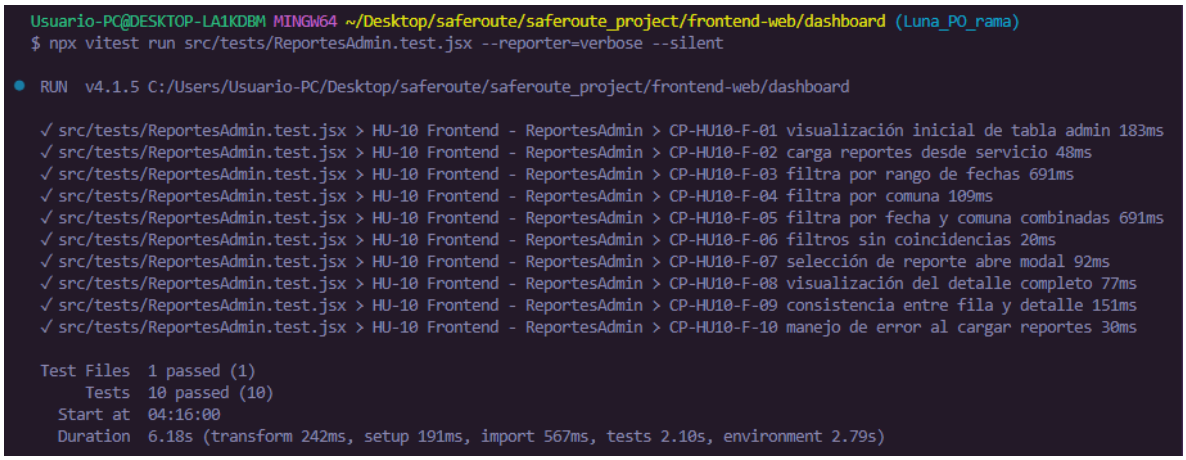


Tabla 12. Casos de prueba HU-10

Identificación	Nombre	Clase/Método	HU	Entrada	Resultado esperado
CP-HU10-F-01	Visualización inicial de tabla admin	ReportesAdmin / render inicial	HU-10	getReportesAdmin() retorna 2 reportes y paginación básica	Primero se muestra Cargando...; luego aparecen las columnas Tipo, Barrio, Comuna, Fecha, Franja y Estado.
CP-HU10-F-02	Carga reportes desde servicio	ReportesAdmin / carga inicial	HU-10	getReportesAdmin() retorna 2 reportes	El componente llama al servicio con { page: 1, limit: 10 } y muestra Centro y San Vicente en la tabla.
CP-HU10-F-03	Filtra por rango de fechas	ReportesAdmin / filtro por fechas	HU-10	Ingresar fechaDesde = 2026-04-01 y fechaHasta = 2026-04-11, luego pulsar Filtrar	El servicio se invoca con { page: 1, limit: 10, fechaDesde: '2026-04-01', fechaHasta: '2026-04-11' }.
CP-HU10-F-04	Filtra por comuna	ReportesAdmin / filtro por comuna	HU-10	Ingresar 1 en Comuna (1-12) y pulsar Filtrar	El servicio se invoca con { page: 1, limit: 10, comuna: '1' }.

CP-HU10-F-05	Filtra por fecha y comuna combinadas	ReportesAdmin / filtros combinados	HU-10	Ingresar comuna = 1, fechaDesde = 2026-04-01, fechaHasta = 2026-04-30 y pulsar Filtrar	El servicio se invoca con { page: 1, limit: 10, comuna: '1', fechaDesde: '2026-04-01', fechaHasta: '2026-04-30' }.
CP-HU10-F-06	Filtros sin coincidencias	ReportesAdmin / listado vacío	HU-10	getReportesAdmin() retorna data: [], total: 0	Se muestra 0 reporte(s) encontrado(s).
CP-HU10-F-07	Selección de reporte abre modal	ReportesAdmin / apertura de detalle	HU-10	Listado con reportes y getReporteById('rep-1') válido; clic sobre Centro	Se llama getReporteById('rep-1') y se muestra el modal Detalle del reporte.
CP-HU10-F-08	Visualización del detalle completo	ReportesAdmin / detalle de reporte	HU-10	Listado con reportes y detalle válido para rep-1; apertura del modal	El modal muestra campos como ID, Tipo reportante, Coordenadas y Descripción.
CP-HU10-F-09	Consistencia entre fila y detalle	ReportesAdmin / consistencia de datos	HU-10	Seleccionar la fila del reporte con barrio Centro	Se abre el detalle y se muestra información coherente con la fila, incluyendo Comuna 1.
CP-HU10-F-10	Manejo de error al cargar reportes	ReportesAdmin / error de carga	HU-10	getReportesAdmin() rechaza con error falló api	Se registra el error en consola mediante console.error.
CP-HU10-B-01	Lista reportes admin con paginación	ReportRepositoryImpl.findForAdmin()	HU-10	Consulta con { page: 1, limit: 10 }; Supabase retorna un reporte y count = 1	El método retorna data con 1 elemento, total = 1, page = 1 y totalPages = 1.
CP-HU10-B-02	Filtra reportes admin por campos enviados	ReportRepositoryImpl.findForAdmin()	HU-10	Filtros tipo_hurto, estado, fechaDesde, fechaHasta y comuna	La consulta aplica eq para tipo_hurto, estado y comuna, además de gte/lte para fecha_incidente.

CP-HU10-B-03	Obtiene detalle por id	ReportRepositoryImpl.findById()	HU-10	Buscar id = 'rep-1'; Supabase retorna un registro válido	El método retorna el reporte con id = 'rep-1' y barrio_ingresado = 'Centro'.
CP-HU10-B-04	Error al listar reportes admin	ReportRepositoryImpl.findForAdmin()	HU-10	Supabase retorna error: { message: 'db error' }	El método lanza Error al obtener reportes admin: db error.
CP-HU10-B-05	Error al consultar detalle por id inexistente	ReportRepositoryImpl.findById()	HU-10	Buscar id = 'rep-x'; Supabase no retorna datos	El método lanza Reporte con id rep-x no encontrado.
CP-HU10-B-06	Controller responde 200 al listar admin	ReportController.listAdmin()	HU-10	req.query = { page: '1', limit: '10' }; repositorio retorna lista paginada	Respuesta 200 con { success: true, data, total, page, totalPages }.
CP-HU10-B-07	Controller responde 200 al obtener detalle	ReportController.getById()	HU-10	req.params.id = 'rep-1'; repositorio retorna { id: 'rep-1' }	Respuesta 200 con { success: true, data: { id: 'rep-1' } }.
CP-HU10-I-01	Flujo integral listado admin con filtros	GET /api/reportes/admin	HU-10	Query con page=1, limit=10, comuna=1, fechaDesde=2026-04-01, fechaHasta=2026-04-11	Respuesta 200, success: true, total = 1, primer resultado con barrio_ingresado = 'Centro', y el repositorio recibe esos filtros como strings.
CP-HU10-I-02	Flujo integral detalle por id	GET /api/reportes/:id	HU-10	Solicitud a /api/reportes/rep-1; repositorio retorna detalle completo	Respuesta 200, success: true, data.id = 'rep-1' y data.descripcion = 'Hurto cerca al parque'.
CP-HU10-I-03	Flujo integral listado sin resultados	GET /api/reportes/admin	HU-10	Query con page=1, limit=10, comuna=99; repositorio retorna lista vacía	Respuesta 200, success: true, data = [] y total = 0.

CP-HU10-I-04	Flujo integral frontend: carga, filtra y abre detalle	ReportesAdmin / flujo completo UI	HU-10	Carga inicial con 2 reportes; luego filtrar por comuna = 1, fechaDesde = 2026-04-01, fechaHasta = 2026-04-11; luego abrir detalle	El componente carga la tabla, vuelve a consultar con los filtros indicados y al seleccionar Centro abre Detalle del reporte con campos como Descripción y Coordenadas.
CP-HU10-I-05	Flujo integral frontend sin coincidencias	ReportesAdmin / flujo de filtrado sin resultados	HU-10	Carga inicial con 2 reportes; luego filtrar por comuna = 9, fechaDesde = 2027-01-01, fechaHasta = 2027-01-31	Tras aplicar filtros, se muestra 0 reporte(s) encontrado(s).

2.3.3. Evaluación

En esta sección, se presentan los resultados obtenidos de realizar los eventos: *Sprint Review y Retrospective.*

- *Sprint Review*

En el Sprint 3 se desarrollaron un total de 4 historias de usuario planificadas en el Sprint Backlog, correspondientes a las funcionalidades de panel administrativo, estadísticas básicas, ranking de zonas con mayor número de hurtos y recuperación de contraseña (HU-10, HU-11, HU-12 y HU-13). De acuerdo con el Definition of Done (DoD) establecido, se logró completar la totalidad de las historias de usuario previstas, cumpliendo con los criterios de aceptación definidos para su finalización.

Durante la revisión, se evidenció un avance significativo en la consolidación del sistema, especialmente en el fortalecimiento del módulo administrativo y la incorporación de herramientas de análisis que permiten consultar información de manera más estructurada. No se presentaron impedimentos significativos que retrasaran la entrega del Sprint; sin embargo, se identificaron dificultades técnicas relacionadas con la tecnología utilizada para el desarrollo del dashboard administrativo, también algunos retos en la implementación del proceso de recuperación de contraseña, por lo que requirió ajustes y mayor esfuerzo de integración.

Como resultado del Sprint Review y con el objetivo de fortalecer la seguridad y el control sobre la información, se incluyó nuevas funcionalidades dentro del Product Backlog y realizar ajustes sobre historias previamente definidas las cuales se pueden observar en la **Tabla 13**.

Tabla 13. Historias de Usuario Nuevas y Ajustadas.

HU Nuevas	HU Ajustadas
HU-19: Gestión y visualización de perfil de usuario Como usuario o administrador, quiero acceder a mi perfil para actualizar mi información personal y configuración de la cuenta, con el fin de mantener mis datos actualizados y gestionar mis preferencias dentro del sistema.	***HU-04: Registro de usuario*** Como usuario, quiero registrarme ingresando mi correo electrónico, apodo y una contraseña válida, aceptando los términos y condiciones, para crear una cuenta y acceder a las funcionalidades del sistema.
HU-20: Gestionar solicitudes de eliminación de reportes Como administrador quiero revisar las solicitudes de eliminación enviadas por usuarios, para decidir si un reporte debe eliminarse definitivamente o restaurarse, garantizando la integridad de la información del sistema.	***HU-18: Gestionar reportes propios*** Como usuario ciudadano, quiero visualizar y editar los reportes de hurto que he registrado, para corregir información y solicitar su eliminación si ya no es válido.
HU-21 : Eliminar cuenta de usuario Como usuario ciudadano, quiero eliminar mi cuenta del sistema, para proteger mi información y dejar de usar la aplicación, asegurando que los reportes realizados no se pierdan y sean reasignados al administrador.	

- *Sprint Retrospective*

En la retrospectiva del Sprint 3 se identificaron oportunidades de mejora relacionadas con el fortalecimiento del conocimiento técnico del equipo y la preparación previa para el uso de tecnologías nuevas y concluyó que es necesario dedicar tiempo a la investigación y validación técnica antes de iniciar desarrollos complejos, especialmente cuando se implementan módulos críticos como paneles administrativos o integraciones avanzadas, además se tomó la decisión de incorporar acciones orientadas a incrementar la confianza del usuario frente al uso y protección de sus datos dentro del sistema, se acordó incorporar nuevas historias de usuario enfocadas en la gestión segura de información, incluyendo

visualización del perfil, solicitudes de eliminación de reportes y eliminación de cuentas, y ajustar historias ya existentes para mejorar su claridad y alinearlas con las necesidades reales del sistema y del usuario.

2.4. Sprint 4

2.4.1. Planeación del *Sprint 4*

Con base en las necesidades identificadas y priorizadas, en el *Sprint Planning Meeting* se seleccionaron las historias de usuario a realizar en el *Sprint*, las cuales se pueden observar en la Tabla 14.

Tabla 14. Sprint backlog

Id	Descripción de la Historia de Usuario	Prioridad	Complejidad
HU04 (Mejora)	Como usuario, quiero registrarme ingresando mi correo electrónico, apodo y una contraseña válida, aceptando los términos y condiciones, para crear una cuenta y acceder a las funcionalidades del sistema.	80	**Alta**
HU14	Como administrador quiero gestionar las cuentas de usuarios (ver, bloquear o reactivar), para mantener el buen uso del sistema y reducir reportes fraudulentos.	36	**Media**
HU15	Como administrador quiero visualizar estadísticas avanzadas (tendencias por zona, tipo de hurto y validación de reportes) para apoyar la toma de decisiones y la coordinación con entidades de seguridad.	36	**Media**
HU16	Como administrador quiero gestionar el estado de los reportes de hurto (mostrar, ocultar o eliminar), para mantener la integridad de la información del sistema y controlar reportes fraudulentos, duplicados o erróneos.	35	**Media**
HU17	Como administrador, quiero editar completamente los informes que hayan sido reasignados tras la eliminación de la cuenta de un usuario, para mantener la calidad, actualización y consistencia de la información registrada en el sistema.	30	**Media**

HU18	Como usuario ciudadano, quiero visualizar y editar los reportes de hurto que he registrado, para corregir información y solicitar su eliminación si ya no es válido.	25	**Media**
HU19	Como usuario o administrador, quiero acceder a mi perfil para actualizar mi información personal y configuración de la cuenta, con el fin de mantener mis datos actualizados y gestionar mis preferencias dentro del sistema.	25	**Media**
HU20	Como administrador quiero revisar las solicitudes de eliminación enviadas por usuarios, para decidir si un reporte debe eliminarse definitivamente o restaurarse, garantizando la integridad de la información del sistema.	16	**Media**
HU21	Como usuario ciudadano, quiero eliminar mi cuenta del sistema, para proteger mi información y dejar de usar la aplicación, asegurando que los reportes realizados no se pierdan y sean reasignados al administrador.	9	**Baja**
HU22	Como administrador quiero descargar reportes de hurtos en formatos Excel, filtrados por fecha, zona y estado de verificación, para realizar análisis externos o respaldos.	6	**Baja**

El *Sprint Goal* establecido fue: *Desarrollar funcionalidades administrativas y de gestión de usuarios que permitan fortalecer el control, seguridad e integridad de la información del sistema, mediante la administración de usuarios y reportes, la gestión de perfiles y solicitudes de eliminación, la incorporación de términos y condiciones en el registro, y la generación de estadísticas y exportación de información para apoyar la toma de decisiones.*

En la Tabla 15, se puede observar el *Definition of Done* (DoF) establecido para este Sprint con las personas responsables por cada tarea.

Tabla 15. Definition of Done del Sprint

Id-HU	**Tarea**	**Responsable**	**Tiempo estimado (HH:MM)**	**Tiempo realizado (HH:MM)**
	Actualizar documentación ER.	**Sarah**	0:00	0:00

	Mejoras Sprint 3	**Sarah**	2:00	3:00
HU14	Especificar la HU-14	**Luna**	0:20	0:10
	Crear tabla auditoria_usuarios (id, admin_id, usuario_id, acción, fecha, detalle opcional).	**Sarah**	0:45	1:30
	Crear índice por estado y fecha_creación para consultas rápidas del panel.	**Sarah**	0:45	1:30
	Definir e implementar endpoint GET /admin/usuarios(listar con paginación y filtros opcionales por estado).	**Fernanda**	1:00	1:00
	Definir punto final PATCH /admin/usuarios/:id/bloquear.	**Fernanda**	1:30	1:00
	Definir punto final PATCH /admin/usuarios/:id/reactivar.	**Fernanda**	1:00	1:00
	Implementar lógicas de bloqueo/reactivación actualizando estado y registrando auditoría.	**Fernanda**	2:00	1:30
	Crear pantalla “Gestión de Usuarios” con tabla y buscador y consumo de endpoint	**Sarah**	0:30	0:20
	Agregar botón de acción Bloquear/Reactivar según estado y consumo de endpoint	**Sarah**	0:30	0:20
	Implementar confirmaciones modal antes de bloquear/reactivar y mensajes de éxito/error	**Sarah**	0:30	0:20
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:30
	Ejecutar casos de prueba	**Luna**	3:00	4:00
HU15	Especificar la HU-15	**Luna**	0:20	0:10
	Definir consultas SQL para obtener estadísticas por: zona (conteo), tipo de hurto (conteo), estado de verificación/validación (conteo)	**Sarah**	0:45	1:30

	Crear vista vw_estadisticas_admin (o varias vistas separadas por tipo de análisis).	**Sarah**	0:45	1:30
	Verificar y optimizar índices por fecha_incidente, zona_id, tipo_hurto, estado_verificacion (si aplica).	**Sarah**	0:45	1:30
	Crear pantalla “Estadísticas Administrativas” dentro del panel con selector de periodo (rango) y Consumir endpoint /admin/estadisticas-avanzadas.	**Sarah**	0:45	3:00
	Renderizar gráficos/tablas (zona, tipo, validación).	**Sarah**	0:45	3:00
	Mostrar mensajes de errores y validaciones	**Sarah**	0:45	3:00
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:30
	Ejecutar casos de prueba	**Luna**	3:00	4:00
HU16	Especificar la HU-16	**Lun**	0:20	0:10
	Verificar que el campo estado exista en tabla reportes (activo / oculto / eliminado).	**Sarah**	0:20	0:10
	Crear tabla auditoria_reportes (id, admin_id, reporte_id, accion, fecha, detalle opcional).	**Sarah**	0:45	1:30
	Crear índice por estado para mejorar consultas del panel.	**Sarah**	0:45	1:30
	Definir endpoint para cambio de estado (ej: PATCH /api/reportes/:id/estado).	**Fernanda**	1:30	1:00
	Implementar lógica para actualizar estado del reporte.	**Fernanda**	2:00	1:30
	Validar permisos (solo admin) y validar valores permitidos del estado.	**Fernanda**	2:00	1:00
	Registrar acción en auditoría (usuario admin, fecha, estado anterior, estado nuevo).	**Fernanda**	2:00	1:00

	Manejo de errores (reporte inexistente, estado inválido).	**Fernanda**	3:00	2:00
	Agregar botones u opción desplegable en el panel para Mostrar / Ocultar / Eliminar.	**Sarah**	0:30	0:20
	Implementar modal de confirmación antes de aplicar el cambio y Consumir endpoint de cambio de estado y refrescar tabla automáticamente.	**Sarah**	0:30	0:20
	Mostrar mensajes de éxito/error.	**Sarah**	0:30	0:20
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:30
	Ejecutar casos de prueba	**Luna**	3:00	3:00
HU17	Especificar la HU-17	**Luna**	0:20	0:10
	Verificar que los campos correspondientes sean editables	**Sarah**	0:20	0:10
	Verificar existencia del campo descripcion como obligatorio para permitir edición.	**Sarah**	0:20	0:10
	Crear tabla auditoria_edicion_reportes (id, admin_id, reporte_id, campos_modificados, fecha, valores_anteriores opcional).	**Sarah**	0:45	1:30
	Definir endpoints y Validar que el reporte tenga descripcion no vacía antes de permitir edición.	**Fernanda**	2:00	1:30
	Implementar actualización solo del campo tipo_hurto y fecha_actualizacion y Registrar el cambio en auditoría	**Fernanda**	3:00	2:20
	Manejar errores (campos inválidos, datos vacíos).	**Fernanda**	2:00	3:00
	Agregar botón “Editar” en detalle del reporte y Mostrar formulario con selector de tipo de hurto.	**Sarah**	0:45	1:15
	Validar que el reporte tenga descripción	**Sarah**	0:45	1:15

	(si no, bloquear botón y mostrar mensaje).			
	Consumir endpoint y actualizar tabla/vista sin recargar todo el panel.	**Sarah**	0:45	1:15
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:30
	Ejecutar casos de prueba	**Luna**	3:00	4:00
HU18	Especificar la HU-18	**Luna**	0:20	0:10
	Verificar campos editables del reporte (fecha, franja, tipo, descripción, objeto hurtado, agresores, etc.).	**Sarah**	0:20	0:10
	Agregar campo estado soportando nuevo estado oculto.	**Sarah**	0:20	0:10
	Crear tabla solicitudes_eliminacion_reportes (reporte_id, usuario_id, motivo, fecha_solicitud, estado_solicitud, revisado_por, fecha_revision) y Relacionar solicitudes con reportes y usuarios (FK).	**Sarah**	0:20	0:15
	Crear índices por usuario_id y estado_solicitud.	**Sarah**	0:20	0:15
	Definir endpoint PUT /api/reportes/:id (editar reporte propio)	**Fernanda**	1:00	1:00
	Definir endpoint POST /api/reportes/:id/solicitar-eliminacion	**Fernanda**	1:00	1:10
	Definir endpoint GET /api/reportes/mis-reportes (solo reportes del usuario autenticado).	**Fernanda**	2:00	1:30
	Definir endpoint GET /api/reportes/mis-reportes/:id (detalle de reporte propio).	**Fernanda**	2:00	1:00
	Reutilizar validaciones del registro de reporte (campos obligatorios, formatos) y definir manejo de errores (reporte no	**Fernanda**	2:00	2:00

	existe, no es dueño, ya solicitado, etc.)			
	Diseñar pantalla “Mis reportes” con listado, filtros básicos, paginación y detalle, y consumir endpoint /mis-reportes y mostrar datos resumidos.	**Sarah**	0:45	1:15
	Implementar botón y formulario de edición con los mismos campos del registro inicial y consumir endpoint PUT para guardar cambios.	**Sarah**	0:45	1:15
	Implementar botón “Solicitar eliminación” con confirmación y campo de motivo.	**Sarah**	0:45	1:15
	Mostrar estado del reporte: activo / oculto / eliminado.	**Sarah**	0:45	1:15
	Mostrar mensajes de éxito/error según respuesta del backend.	**Sarah**	0:30	1:00
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:30
	Ejecutar casos de prueba	**Luna**	3:00	4:00
HU19	Especificar la HU-19	**Luna**	0:20	0:10
	Crear índice por usuario_id en configuracion_alertas.	**Sarah**	0:45	1:30
	Implementar endpoint PUT /api/perfil para actualizar: username (apodo), foto_url	**Fernanda**	1:00	1:00
	Implementar endpoint PUT /api/perfil/password para cambio de contraseña.	**Fernanda**	3:00	2:00
	Implementar endpoint PUT /api/perfil/notificaciones para activar/desactivar notificaciones.	**Fernanda**	2:00	2:00
	Implementar validaciones y manejar errores y respuestas estándar.	**Fernanda**	2:00	1:30
	Crear pantalla y consumo de endpoint	**Sarah**	0:45	0:30

	para“Perfil Admin” con: correo, apodo, foto, fecha creación, opción de cambiar contraseña, botón cerrar sesión			
	Mostrar mensajes de confirmación.	**Sarah**	0:45	0:30
	Crear pantalla y consumo de endpoint para “Perfil” con: correo, apodo, foto, fecha creación, opción de cambiar contraseña, switch notificaciones y ubicación activas, botón cerrar sesión	**Sarah**	0:45	0:30
	Mostrar mensajes de advertencia al desactivar notificaciones y ubicación (texto informativo).	**Sarah**	0:45	0:30
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:30
	Ejecutar casos de prueba	**Luna**	3:00	4:00
HU20	Especificar la HU-20	**Luna**	0:20	0:10
	Crear tabla solicitudes_eliminacion con campos: id, reporte_id (FK), usuario_id (FK solicitante), estado_solicitud (pendiente/aprobada/rechazada), fecha_solicitud, fecha_resolucion, admin_id (FK quien resolvió), motivo (opcional)	**Sarah**	0:20	0:15
	Crear índices por estado_solicitud y reporte_id para mejorar consultas.	**Sarah**	0:20	0:15
	Ajustar relación para que un reporte no tenga múltiples solicitudes pendientes simultáneamente.	**Sarah**	0:20	0:15
	Definir endpoint GET /admin/solicitudes-eliminacion (listar solicitudes pendientes).	**Fernanda**	1:00	1:00
	Definir endpoint GET /admin/solicitudes-eliminacion/:id (detalle).	**Fernanda**	1:00	1:00
	Definir endpoint POST /admin/solicitudes-eliminacion/:id/apro	**Fernanda**	1:30	1:20

	bar.			
	Definir endpoint POST /admin/solicitudes-eliminacion/:id/rechazar.	**Fernanda**	1:00	1:00
	Implementar validaciones	**Fernanda**	1:30	2:00
	Diseñar pantalla “Solicitudes de eliminación”.	**Sarah**	1:00	0:50
	Implementar tabla/listado con filtros básicos (pendiente/aprobada/rechazada) y vista de detalle del reporte solicitado.	**Sarah**	1:00	0:45
	Implementar botones: “Aprobar eliminación” “Rechazar solicitud”	**Sarah**	1:00	0:35
	Mostrar confirmación antes de ejecutar la acción.	**Sarah**	1:00	0:35
	Mostrar mensajes de éxito/error	**Sarah**	1:00	0:40
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:30
	Ejecutar casos de prueba	**Luna**	3:00	4:00
HU21	Especificar la HU-21	**Luna**	0:20	0:10
	Ajustar la tabla usuarios para que el campo estado permita valores: activo, bloqueado, eliminado.	**Sarah**	0:45	1:30
	Definir en BD o configuración del sistema el admin_id fijo para reasignación de reportes.	**Sarah**	0:45	1:30
	Crear/validar índice por usuario_id en reportes para reasignación rápida.	**Sarah**	0:45	1:30
	Definir endpoint PATCH /api/usuarios/eliminar-cuenta (usuario autenticado).	**Fernanda**	2:00	2:00
	Invalidar token/sesión del usuario automáticamente.	**Fernanda**	4:00	3:30
	Agregar opción y consumo de endpoint	**Sarah**	0:30	0:10

	para “Eliminar cuenta” en pantalla Perfil.			
	Implementar modal de confirmación (sí/no) y mensaje final de cuenta eliminada que redirige a login.	**Sarah**	1:00	0:40
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:30
	Ejecutar casos de prueba	**Luna**	3:00	5:00
HU22	Especificar la HU-22	**Luna**	0:20	0:10
	Limpieza de BD	**Sarah**	0:45	1:30
	Definir vista opcional vw_export_reportes_admin para estandarizar datos de exportación.	**Sarah**	0:45	1:30
	Definir endpoint de exportación con middleware: GET /api/reportes/admin/export?fechaDesde=&fechaHasta=&zona=&estado=&formato=csv|excel y reutilizar lógica de filtros	**Fernanda**	3:00	3:00
	Implementar generación de archivo (CSV, Excel), implementar control de tamaño: limitar número máximo de registros exportados, retornar mensaje si excede el límite y retornar archivo como descarga (Content-Disposition).	**Fernanda**	3:00	3:00
	Agregar botón “Descargar reportes” en el panel de reportes y consumo de endpoint	**Sarah**	0:20	0:20
	Implementar selector de formato (CSV / Excel).	**Sarah**	0:45	0:20
	Enviar filtros actuales al endpoint de exportación.	**Sarah**	0:35	0:20
	Mostrar indicador de carga/progreso mientras se genera el archivo.	**Sarah**	0:30	0:20
	Descargar archivo automáticamente al	**Sarah**	0:10	0:10

	completar.			
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:30
	Ejecutar casos de prueba	**Luna**	3:00	2:00
HU04 (Mejora)	Especificar la HU-04	**Luna**	0:20	0:10
	Crear campo o tabla para almacenar evidencia de aceptación de términos (fecha/hora y versión).	**Sarah**	0:20	0:15
	Definir versión actual del documento de términos y condiciones (ej. v1.0).	**Sarah**	0:20	0:15
	Crear migración SQL para guardar aceptación de términos en el registro.	**Sarah**	0:20	0:15
	Actualizar endpoint POST /auth/register para validar aceptación obligatoria de términos.	**Fernanda**	1:30	1:00
	Guardar evidencia de aceptación (fecha/hora y versión) asociada al usuario creado.	**Fernanda**	1:00	1:00
	Definir endpoint o archivo estático para consultar términos y condiciones (opcional: GET /legal/terminos).	**Fernanda**	1:00	1:00
	Actualizar pantalla “Registrarse” agregando checkbox obligatorio de aceptación y consumir endpoints.	**Sarah**	1:00	0:45
	Agregar enlace para visualizar “Términos y condiciones”.	**Sarah**	1:00	0:45
	Bloquear registro si no se marca la casilla y mostrar mensaje de validación si el usuario no acepta los términos.	**Sarah**	1:30	0:45
	Identificar escenarios de prueba	**Luna**	1:00	0:10
	Diseñar y crear casos de prueba	**Luna**	2:00	0:30
	Ejecutar casos de prueba	**Luna**	3:00	2:00

2.4.2. Ejecución del *Sprint 4*

A continuación, se presenta una síntesis de los resultados obtenidos en la ejecución de las tareas establecidas para el *DoD*.

- **Especificación de la Historia de Usuario**

En el enlace indicado ESPECIFICACIÓN 4, se puede observar la especificación completa de las historias de usuario seleccionadas para el Sprint 4, correspondientes a funcionalidades de administración de usuarios, gestión de reportes, configuración de perfil, control de solicitudes y seguridad de la información. Adicionalmente, en este Sprint se incorporó una mejora a la HU-04 relacionada con la aceptación de términos y condiciones durante el proceso de registro de usuarios, fortaleciendo el manejo y consentimiento del uso de la información dentro del sistema. En la Tabla 16, se presenta a manera de ejemplo la especificación correspondiente a la HU-14, la cual describe el proceso de administración de usuarios y permisos dentro del sistema, incluyendo las reglas de negocio, requisitos no funcionales y flujo básico del proceso.

Tabla 16. Ejemplo de la especificación de la HU-14

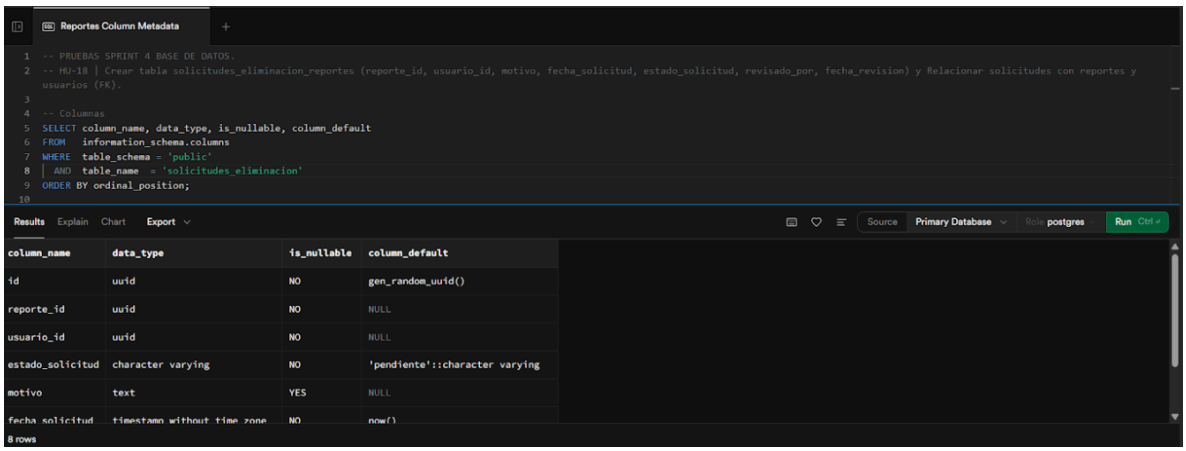
Historia de Usuario			
Código:	**HU-14**		
Nombre:	**Administrar usuarios y permisos**		
Actor:	**Administrador**		
Descripción	Como administrador quiero gestionar las cuentas de usuarios (ver, bloquear o reactivar), para mantener el buen uso del sistema y reducir reportes fraudulentos.		
Criterios de aceptación:	**CID**	**Condición**	**Resultado**
	1	El administrador accede a la sección de “Usuarios”.	El sistema muestra una tabla con usuarios (ID, correo, rol, estado: activo/bloqueado, fecha de registro).
	2	El administrador selecciona un usuario y elige la opción “bloquear”.	El sistema cambia el estado del usuario a “bloqueado” e impide que pueda iniciar sesión o registrar nuevos reportes.
	3	El administrador selecciona un usuario bloqueado y elige la opción “reactivar”.	El sistema cambia el estado a “activo” y registra el cambio en el log de auditoría.

- **Diseño e implementación de la base de datos**

En el Sprint 4 la base de datos se enfocó en fortalecer para que la administración del sistema sea mucho más eficiente y transparente. Agregamos un registro de auditoría para seguirle la pista a cualquier cambio, bloqueo o reactivación de usuarios y reportes. También renovamos la gestión de contenidos: los reportes ahora pueden estar activos, ocultos o eliminados, y diseñamos el flujo para que la administración revise las solicitudes de borrado que envían los usuarios. Por último, optimizamos los índices de búsqueda para acelerar las consultas en el panel de control y sumamos el soporte necesario para ajustar perfiles y guardar la aceptación de términos y condiciones durante el registro.

En la **Figura 18** se presenta como ejemplo la estructura correspondiente a la tabla de solicitudes de eliminación de reportes implementada para la **HU-18 y HU-20**. En el enlace indicado BASE DE DATOS 4, se puede consultar el diagrama actualizado y el código completo de las tablas, relaciones, índices y migraciones implementadas durante el Sprint 4.

Figura 18. Tabla solicitudes_eliminacion_reportes



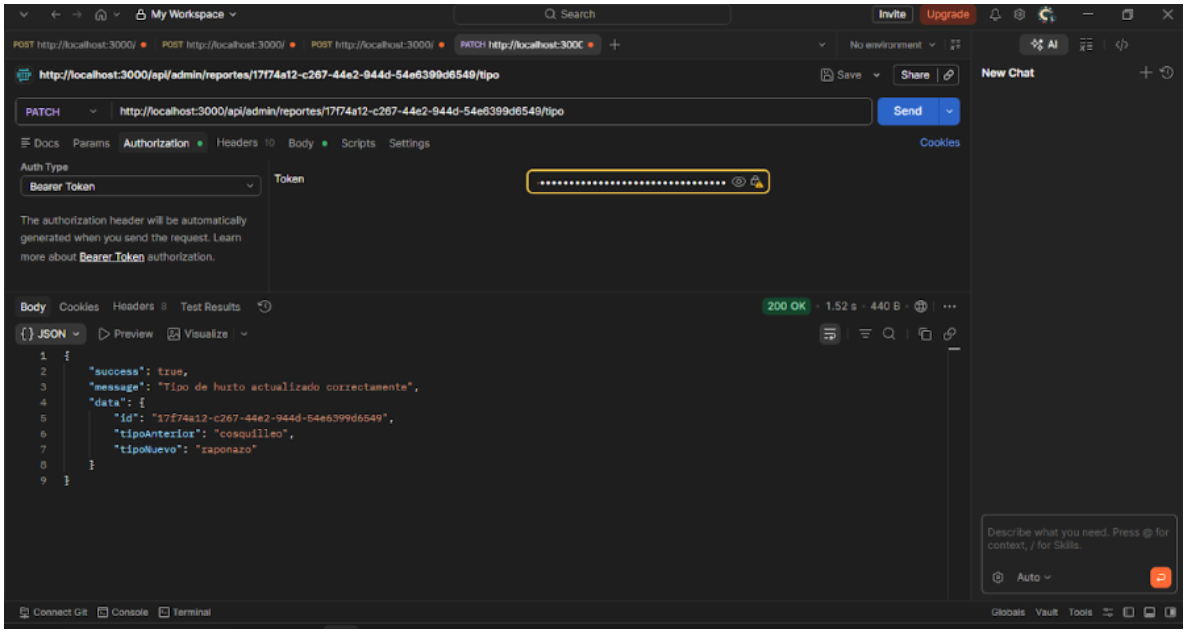
- **Desarrollo del Back-end**

Durante el Sprint 4 se desarrollaron e implementaron endpoints para la administración de usuarios, la gestión de reportes y el control de solicitudes, protegidos mediante autenticación y roles. Entre las principales funcionalidades destacan el bloqueo y reactivación de usuarios, la edición restringida del tipo de hurto (HU-17) con registro en auditoría, la gestión de perfiles, la eliminación de cuentas, la exportación de reportes a CSV y Excel, y la validación obligatoria de términos y condiciones en el registro (HU-04).

En la **Figura 19** se presenta la evidencia del funcionamiento del endpoint correspondiente a la edición del tipo de hurto implementado para la **HU-17**. En el enlace indicado

BACKEND 4, se encuentran las demás evidencias de pruebas realizadas a los endpoints desarrollados durante el Sprint 4.

Figura 19. Prueba del endpoint EditarTipoHurto – HU-17



- **Diseño de la Interfaz**
  - *Desarrollo del Front-end Flutter (Usuario)*

Durante el Sprint 4 se desarrollaron nuevas interfaces móviles para la gestión de reportes propios (HU-18), perfil (HU-19) y eliminación de cuenta (HU-21), asimismo en la HU-18, el usuario puede listar, editar y solicitar la eliminación de sus reportes, cambiando temporalmente su estado a "oculto" para validación administrativa. También la HU-19 permitió actualizar datos personales, preferencias y cerrar sesión, mientras que la HU-21 aseguró el cierre de sesión y la reasignación administrativa de reportes al eliminar la cuenta. Además, la HU-04 añadió la aceptación obligatoria de términos y condiciones en el registro.

En la **Figura 20** se presenta la pantalla correspondiente a la solicitud de eliminación de reportes implementada para la **HU-18**. En el enlace indicado FRONTEND MOVIL 4, se encuentran las demás pantallas desarrolladas durante el Sprint 4.

Figura 20. Pantalla de solicitud de eliminación de reportes – HU-18 (Flutter)



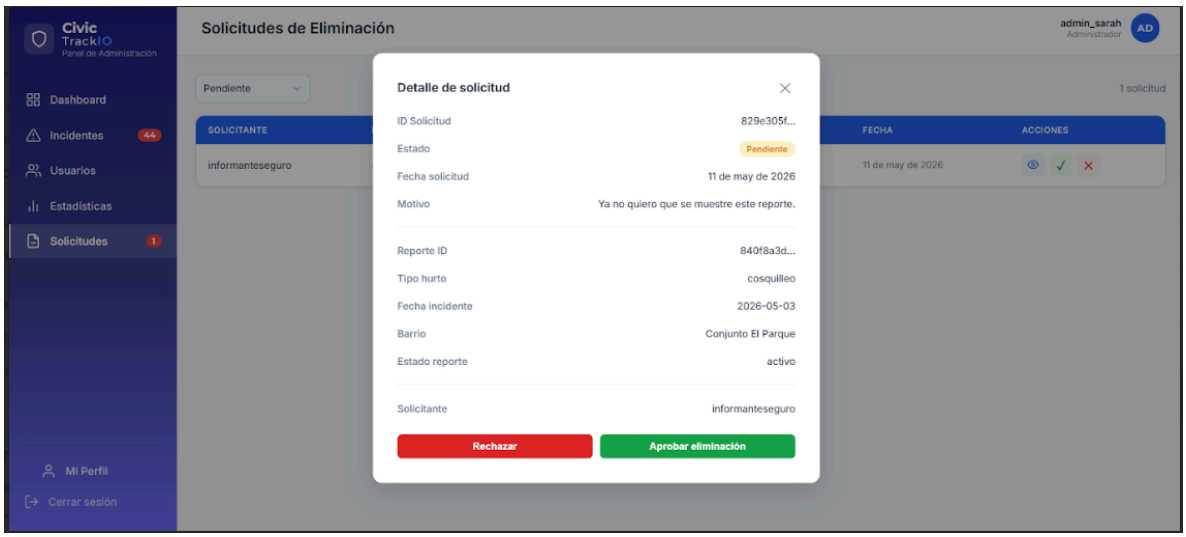
- *Desarrollo del Front-end React (Administrador)*

También se desarrollaron nuevas interfaces web del panel administrativo para la gestión de usuarios (HU-14), estadísticas avanzadas (HU-15) y estado de reportes (HU-16), incluyendo gráficos, filtros dinámicos y acciones para administrar su visibilidad. Además, se añadió la edición del tipo de hurto con auditoría (HU-17).

También, integró la gestión de solicitudes para eliminar reportes (HU-20), permitiendo al administrador aprobarlas o rechazarlas tras revisar su detalle. Finalmente, se sumaron funcionalidades del perfil administrativo (HU-19) y la exportación de reportes a formatos externos (HU-22) para facilitar su análisis fuera del sistema.

En la **Figura 21** se presentará la evidencia correspondiente a la pantalla de solicitudes de eliminación de reportes implementada para la **HU-20**. En el enlace indicado FRONTEND WEB 4, se encuentran las demás interfaces desarrolladas durante el Sprint 4.

Figura 22. Pantalla de solicitudes de eliminación de reportes – HU-20 (React)



- **Pruebas y Validación**

Durante el Sprint 4 se diseñaron y ejecutaron escenarios de prueba para las funcionalidades implementadas en las historias de usuario HU-14, HU-15, HU-16, HU-17, HU-18, HU-19, HU-20, HU-21 y HU-22, así como para la mejora aplicada a la HU-04 relacionada con términos y condiciones. Se validaron casos exitosos y fallidos relacionados con la gestión de usuarios, administración de reportes, edición de información, solicitudes de eliminación, configuración de perfil, descarga de reportes administrativos y validaciones de aceptación obligatoria durante el registro de usuarios.

En la **Tabla 17** se presentan los casos de prueba correspondientes a la **HU-22**. En la **Figura 23** se evidencia la ejecución de las pruebas realizadas para la funcionalidad de descarga de reportes administrativos. Finalmente, en el enlace indicado PRUEBAS Y VALIDACIONES, se encuentran disponibles los casos de prueba completos y evidencias del Sprint 4.

Figura 23. Evidencia de ejecución de pruebas – HU-22



Tabla 17. Casos de prueba HU-22

Identificación	**Nombre**	**Clase/Método**	**HU**	**Entrada**	**Resultado esperado**
CP-HU21-F-01	Renderiza opción de descarga en panel de informes	TabIncidentes/ botón de descarga frontend	HU-21	Carga inicial de la tabla de informes	Se muestra el botón Descargary el botón Limpiarjunto al listado de informes.
CP-HU21-F-02	Descarga CSV con filtros aplicados	TabIncidentes/ Exportar interfaz CSV	HU-21	Usuario selecciona estado = "activo", comuna = "5", formato = "csv", fechas 2026-05-01a 2026-05-31y hace clic enDescargar	Se llama a GET /api/admin/reportes/exportcon { formato: "csv", fechaDesde: "2026-05-01", fechaHasta: "2026-05-31", estado: "activo", zona: "5", responseType: "blob" }, se crea una URL blob, se dispara la descarga y se muestra

					Archivo descargado correctamente.
CP-HU21-F-03	Muestra error si exportación devuelve respuesta no descargable	TabIncidentes/ error de interfaz de exportación	HU-21	API responde error 400con mensajeNo hay registros para exportar con los filtros aplicados.	Se muestra el mensaje de error y no se dispara la descarga (anchor.clickno se llama).
CP-HU21-F-04	Muestra indicador de progreso durante la exportación	TabIncidentes/ estado de carga frontend	HU-21	El usuario hace clic en Descargary la API responde de forma diferente	El botón de descarga permanece visible/deshabilitado durante la carga, indicando progreso hasta que llega la respuesta.
CP-HU21-F-05	Error de pantalla si falla la exportación	TabIncidentes/ error genérico frontend	HU-21	API rechaza conError("fallo")	Se muestra Error al exportar.
CP-HU22-B-01	Exporta CSV con filtros válidos	adminController.exportarReportes()	HU-22	query = { fechaDesde: "2026-05-01", fechaHasta: "2026-05-31", zona: "5", estado: "activo", formato: "csv" }	Se consulta vw_export_reportes_admincon select("*"), limit(5001), filtros gte, lte, eqpor fecha, estado y comuna; se responden encabezados Content-Type: text/csv;

					charset=utf-8y Content-Disposition: attachment; filename="reportes_*.csv", y se envía CSV con cabecera reporte_id,fecha_incidente,....
CP-HU22-B-02	Exporta Excel con filtros válidos	adminController.exportarReportes()	HU-22	query = { formato: "excel" }; datos válidos	Se generan encabezados Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheety Content-Disposition: attachment; filename="reportes_*.xlsx", se escribe el libro de trabajo con exceljsy se finaliza la respuesta.
CP-HU22-B-03	Rechaza formato inválido	adminController.exportarReportes()	HU-22	query = { formato: "pdf" }	Respuesta 400con { success: false, message: "formato debe ser 'csv' o 'excel'" }.
CP-HU22-B-04	Responde mensaje cuando no haya datos	adminController.exportarReportes()	HU-22	Consulta retornodata = []	Respuesta 200con { success: true, message: "No hay registros para exportar con los filtros aplicados." }.

CP-HU22-B-05	Rechaza exportación si excede límite	adminController.exportarReportes()	HU-22	Consulta retorno 5001registros	Respuesta 400con { success: false, message: "La consulta excede el límite de 5000 registros. Aplica más filtros para reducir los resultados." }.

2.4.3. Evaluación

En esta sección, se presentan los resultados obtenidos de realizar los eventos: *Sprint Review y Retrospective.*

- *Sprint Review*

En el Sprint 4 se desarrollaron un total de nueve historias de usuario y una mejora funcional planificadas en el Sprint Backlog, correspondientes a las funcionalidades de administración de usuarios, estadísticas avanzadas, gestión de reportes, perfil de usuario, solicitudes de eliminación, eliminación de cuentas, exportación de reportes y la incorporación de términos y condiciones en el proceso de registro (HU-14, HU-15, HU-16, HU-17, HU-18, HU-19, HU-20, HU-21, HU-22 y mejora de la HU-04). De acuerdo con el Definition of Done (DoD) establecido, se logró completar la totalidad de las funcionalidades previstas, cumpliendo los criterios de aceptación definidos y alcanzando una Velocity del 100 % y un Índice de Valor Entregado (IVE) igual a 1, lo que evidencia que la entrega estuvo alineada con el valor esperado para el Sprint.

En esta etapa, el trabajo se enfocó en fortalecer la administración, el control de seguridad y la privacidad de la información del sistema. Se implementaron herramientas clave para que los administradores puedan gestionar cuentas de usuarios mediante bloqueos o reactivaciones, visualizar estadísticas detalladas sobre las tendencias de hurtos y exportar reportes en formatos ejecutivos como Excel. Asimismo, se facilitó a los ciudadanos el control sobre sus datos al permitirles actualizar su perfil, aceptar términos y condiciones, e incluso solicitar la eliminación de sus publicaciones o de sus cuentas personales.

Para responder a las observaciones recibidas durante la revisión con los stakeholders, se realizaron ajustes inmediatos en la gestión de permisos y la auditoría del sistema. Se definió una política clara donde el administrador solo puede modificar aspectos parciales en los reportes de usuarios activos, pero adquiere control total sobre los informes cuando una cuenta es eliminada y el reporte le es reasignado, permitiendo corregir incluso el tipo de

delito registrado. Todo este proceso quedó respaldado con un registro detallado en el historial de cambios, lo que permitió cerrar el ciclo de desarrollo garantizando la estabilidad, la trazabilidad y la integridad de la plataforma.

- *Sprint Retrospective*

Durante la retrospectiva del Sprint 4, el equipo concluyó que la planificación realizada permitió cumplir satisfactoriamente con los objetivos propuestos y finalizar la totalidad de las funcionalidades contempladas para el proyecto. Se fortalecieron las competencias técnicas relacionadas con la implementación de módulos administrativos, la gestión de accesos basada en roles y la incorporación de mecanismos de auditoría para garantizar la integridad y trazabilidad de los datos.

Asimismo, se evidenció la importancia de mantener una alineación constante entre las historias de usuario, los requerimientos del sistema y la implementación real, por esto, la retroalimentación recibida por parte de los stakeholders fue clave para identificar y ajustar a tiempo aspectos de la lógica de negocio, como la reasignación de reportes y los permisos diferenciados del administrador. Esta experiencia resaltó la necesidad de validar de forma continua los avances con los usuarios e interesados, asegurando que la solución responda a las necesidades reales del contexto.

Finalmente, el equipo reconoció el valor de ejecutar pruebas funcionales de manera constante a lo largo de cada iteración. Afrontar los desafíos técnicos de manera colaborativa no solo permitió elevar la calidad de las entregas antes del cierre definitivo, sino que también dejó aprendizajes significativos en el manejo de entregables y en el refinamiento continuo del producto final.